

Credit Card Fraud Detection and Risk Assessment

Akhil Gotkhindi BS Mechanical Engineering (2021)
Masahiro Uchida BS Computer Science and Mathematics (2021)
April 19, 2026

Department of Electrical and Computer Engineering
Northeastern University

ABSTRACT

This project investigates machine learning approaches for two financial risk assessment tasks: credit card fraud detection and credit risk prediction. Both problems present significant challenges due to class imbalance and unequal misclassification costs. For fraud detection, we evaluate Logistic Regression, Support Vector Machine (SVM), and Random Forest models using a highly imbalanced transaction dataset, while for credit scoring, we apply Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, and a Stacked Ensemble on the German Credit Dataset.

To address class imbalance, Synthetic Minority Over-sampling Technique (SMOTE) was applied to training set within cross-validation folds to prevent data leakage. Model performance was evaluated using stratified cross-validation. In addition to standard metrics such as precision, recall and ROC_AUC, cost-sensitive evaluation and decision threshold tuning based on model outputs was incorporated.

Results show that SVM achieved high recall in fraud detection, while for credit scoring, XGBoost achieved the highest cross-validated accuracy of 75% and the Stacked Ensemble achieved the lowest business cost and highest recall, demonstrating ensemble superiority. Decision threshold tuning further reduced business costs by 18%.

These findings show the importance of proper validation, cost-sensitive evaluation, and threshold selection when constructing machine learning models in financial tasks.

CONTENTS

I	Introduction.....	6
II	Datasets.....	7
A	Credit Card Fraud Dataset.....	7
B	German Credit Dataset.....	7
C	Exploratory Data Analysis.....	8
1)	Credit Card Fraud Dataset	8
2)	German Credit Dataset	9
D	Preprocessing.....	9
1)	Credit Card Fraud Dataset	9
2)	German Credit Dataset	10
III	Methodology.....	11
A	Problem Formulation	11
B	Models Used	11
1)	Logistic Regression Model	12
2)	Support Vector Machine Model	12
3)	Decision Tree Model.....	13
4)	Random Forest Model.....	14
5)	Gradient Boosting Model	14
6)	XGBoost Model	15
7)	Stacked Ensemble	16
C	Feature Selection.....	16
D	Handling Class Imbalance	17
E	Validation Strategy	18
1)	Credit Card Fraud Dataset	18
2)	German Credit Dataset	19
F	Evaluation Metrics.....	20
1)	Credit Card Fraud Dataset	20
2)	German Credit Dataset	22
IV	Results.....	22
A	Credit Card Fraud Dataset.....	22

B	German Credit Dataset.....	29
V	Discussions.....	32
VI	Conclusions.....	34
	References.....	35
	Vita.....	35
	Appendix.....	36

I INTRODUCTION

Financial institutions face two critical challenges, such as detecting fraudulent credit card transactions and accurately assessing credit risk for loan applicants. Credit card fraud could result in billions of dollars of annual loss globally, while poor credit risk assessment could lead to loan defaults that threaten financial stability. Traditional rule-based systems struggle with the complexity and evolving nature of those problems. Machine learning offers powerful tools to address these challenges through pattern recognition, adaptive learning, and sophisticated handling of imbalanced datasets.

This project will explore two financial machine learning tasks: detecting fraud in credit card transactions and predicting risk for credit default. Both tasks are formulated as binary classification problems, where goal is to accurately detect fraud transactions and high-risk instances based on observed features.

There are several challenges to these problems. First, fraud detection datasets are highly imbalanced, where only small proportion of data are fraud samples, which can bias models to classify most samples as non-fraud. Additionally, misclassification costs are not equal. Failure to detect fraud transactions or high credit risks have significant consequences compared to misclassifying non-fraud as fraud or low-risk as high-risk. Also, proper validation strategies are required to evaluate models without data leakage.

This project was inspired by prior research exploring similar problems. The article, *Credit Card Fraud Data Analysis and Prediction Using Machine Learning Algorithms* [1], demonstrates the effectiveness of various classification algorithms for fraud detection, highlighting the importance of handling class imbalance and evaluating models mainly using recalls. Similarly, the article, *A machine learning-based credit risk prediction engine system using a stacked classifier and a filter-based feature selection method* [2], describes an ensemble-based approach for credit risk prediction.

Additional resources were studied to address challenge of handling class imbalance. The article, *SMOTE: Synthetic Minority Over-sampling Technique* [3], introduced SMOTE, a widely used method for generating synthetic minority samples to temporally make datasets balanced. Also, the article, *ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning* [4], introduced another method which is an extension of SMOTE, which focuses on more instances that are difficult to learn. These techniques are standard tools for improving model performance on imbalanced classification problems.

This project evaluates multiple machine learning models, with Logistic Regression as a baseline. Class imbalance is handled using SMOTE, and model performance is evaluated using cross-validation.

The remainder of this report is organized as follows. Section II describes the datasets. Section III presents methodology including machine learning methods, validation strategies, and evaluation methods. Section IV describes the results, section V provides discussions with future directions, and section VI provides conclusions.

II DATASETS

Two datasets were imported from Kaggle. One is the dataset for the credit card fraud detection task, and another one is for the credit risk assessment task. Both datasets are analyzed and preprocessed before any machine learning techniques are applied. Datasets are also split into training and test datasets, where training datasets are used to train, tune, and compare models, and test datasets are used to evaluate final model performance.

A *Credit Card Fraud Dataset*

The dataset used for the fraud detection task is the tabular dataset which contains data about credit card transactions. The dataset is downloaded from Kaggle and contains information of 284,807 transactions. Each sample has 30 features. However, two of them have actual meaning to it: “Time” and “Amount”, where other features are labeled as “V1”, “V2”, to “V28”, which are latent features obtained by PCA transformation, due to the privacy concerns. Each sample of transactions are labeled with “Fraud” or “Non-fraud”.

B *German Credit Dataset*

The dataset used for the credit risk assessment task is the tabular dataset containing information on 1,000 loan applications, obtained from the UCI Machine Learning Repository (Statlog German Credit Data). Each sample represents a loan applicant described by a set of attributes and is labeled as either good credit risk or bad credit risk, making this a binary classification problem. The dataset contains 700 good credit samples and 300 bad credit samples, resulting in a 70:30 class ratio.

The dataset originally contains 20 encoded categorical and numeric attributes. To improve interpretability, nine attributes were selected and decoded from UCI encoding into human-readable form. These attributes are: Age (numeric, in years), Sex (male/ female), Job (ordinal, 0 to 3, representing unskilled non-resident to highly skilled), Housing (own/ rent/ free), Saving accounts (little/ moderate/ quite rich/ rich), Checking account (little/moderate/quite rich), Credit amount (numeric, in Deutsche Mark), Duration (numeric, in months), and Purpose (car / education/ furniture/ radioTV/

repairs/ business/ vacation). Two features, Saving accounts and Checking account, contained missing values in 183 (18.3%) and 394 (39.4%) of samples respectively, due to applicants having no such account.

C Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed for both datasets to understand nature of the data and decide ways to preprocess them before applying machine learning algorithms.

1) Credit Card Fraud Dataset

For the fraud dataset, all features are numerical, and labeling feature is binary where 0 represents non-fraud transactions, and 1 represents fraud transactions. No samples from this dataset contain missing values. However, the dataset contains 284,315 samples of non-fraud transactions, but only 492 samples of fraud transactions. So, there is a heavy class imbalance which indicates requirements for some adjustments so that the machine learning models will not be biased to classify most samples as non-fraud for better accuracy.

Distribution of two non-latent numerical features are visualized as histogram, (See Appendix A-1 is for “Amount” features, and Appendix A-2 is for “Time features”). Distributions of features for only non-fraud samples and only fraud samples are plotted as well as distributions for all samples, but distributions between fraud and non-fraud samples are similar.

Distributions for the transaction amount (feature name “Amount”) were heavily skewed, where most samples have transaction amount of below 1000, and only few samples have transaction amount of above 5000, and there are few samples with extremely high transaction amounts such as 20,000. The skewness is also shown on the box plot (See Appendix A-3), where most visible points are displayed as outliers.

This behavior is expected, because although transaction amount can take any value, most transactions are done within low amount such as for everyday purchase, but there are few occasions where large amount of transactions are required. So, the distribution represents the legitimate transaction behavior, which is natural and is not data errors. Therefore, some special transformation must be applied to “Amount” feature to improve model stability.

On the other hand, distributions for the time for transaction (feature name “Time”) were close to uniform, and according to its box plot (See Appendix A-3), there are no obvious skews and no outliers were observed.

Finally, correlation maps for all features in the fraud dataset are plotted (See Appendix A-4). According to the correlation maps, all latent features have correlations very close to 0. For named features, although there are some latent features which might be related to them, there were no pairs of features which absolute correlations exceeding 0.5. Therefore, no features are highly correlated with each other, so dimensionality reduction is not needed.

2) *German Credit Dataset*

Exploratory Data Analysis was performed on the German Credit dataset to understand the distribution of features and their relationship to credit risk, and to inform preprocessing decisions.

The dataset exhibits moderate class imbalance with a 70:30 ratio of good to bad credit applicants. Bad loan applicants tend to request larger credit amounts, with a higher median and wider spread compared to good applicants. Loan duration also differs significantly between classes, with bad loans having longer repayment periods. Age distributions for both classes peak around 25 to 30 years, but bad credit risk skews toward younger applicants.

Among categorical features, checking account status emerged as the most informative predictor. Applicants with no checking account represent the largest risk group. The missing value analysis confirmed that 39.4% of applicants lacked a checking account record and 18.3% lacked a savings account record, suggesting that account absence itself carries predictive signal and should not be treated as noise. A correlation analysis showed no severe multicollinearity among the selected features, so no further dimensionality reduction was required beyond the Information Gain based feature selection.

D *Preprocessing*

Based on observations from exploratory data analysis, column-wise transformations are applied to each dataset to ensure that models will not be biased by feature distributions. Then, datasets were split into training datasets and test datasets, where training datasets are further split by cross-validation, and used for model training, evaluation, and comparison, while test datasets are used for final model evaluation.

1) *Credit Card Fraud Dataset*

To address the extreme right-skew of the “Amount” feature in the fraud dataset, log transformation was applied to it. The log transformation is applied by following equation:

$$x_{new} = \log(1 + x)$$

, where x is the vector which contains original feature values, and x_{new} is the features values after the transformation was applied. Log was applied to reduce the skewness by shrinking outliers to lower values, reducing the effect of outliers, and 1 was added to avoid log of 0. The equation was converted to function which can be applied as column transformation using FunctionTransformer object from scikit-learn.

After log transformation was applied to “Amount” feature, standard scaling was applied to both non-latent numerical features so they will have 0 mean and standard deviation of 1. The equation for standard scaling is the following:

$$x_{new} = \frac{x - \mu}{\sigma}$$

, where μ is the feature mean and σ is the standard deviation of the feature. Note that these are mean and standard deviation of training set, and validation and test sets are transformed using same parameters. Since all latent features were already standardized, this transformation was only applied to “Time” and log-transformed “Amount” features, and all latent components are untouched.

All transformations are applied using the ColumnTransformer object from scikit-learn. This object stores mapping of transformation function and list of columns, and transforms values of pandas DataFrame by applying given transformation function to corresponding columns. The transformation function to “Amount” column was created using Pipeline object, also from scikit-learn, which takes sequences of functions to be applied to that column during the transformation. (See appendix A-5 for code for transformation pipeline for fraud dataset.)

The dataset was split into training and test datasets using “train_test_split” object from scikit-learn, with 20% of samples became test dataset and remaining 80% of samples became training dataset. The class ratio was preserved by setting “stratify” parameters, and indeed, 0.17% of both training set and test set are fraud samples, which is also same ratio as the original dataset (See appendix A-6).

2) *German Credit Dataset*

Start Based on observations from exploratory data analysis, a sequence of transformations was applied to the German Credit dataset before model training.

Missing values in Saving accounts and Checking account were filled with the category label “none” rather than imputed or dropped. This preserves the information that an applicant has no such account, which is a distinct and meaningful risk indicator separable from having a small account balance.

Categorical features were converted to numeric representations using one-hot encoding. For a categorical feature with k distinct values, $k - 1$ binary indicator columns were created, with the first category dropped per feature to avoid multicollinearity. This expanded the feature space from 9 to 22 columns. Feature selection was then applied to reduce the 22 encoded features to the 7 most informative. Standard scaling was applied after feature selection. For each feature, the transformation is:

$$x_{new} = \frac{x - \mu}{\sigma}$$

, where μ is the mean and σ is the standard deviation of the training set. Scaling parameters were fitted exclusively on the training set and applied to both training and test sets to prevent data leakage. The dataset was split into training and test sets using stratified sampling, with 80% (800 samples) assigned to training and 20% (200 samples) assigned to test, preserving the 70:30 class ratio in both partitions. All plots for exploratory data analysis are in Appendix B-1.

III METHODOLOGY

This section addresses mathematical settings for two binary classification tasks: fraud detection and risk assessment, introduce models which are used for these tasks, describe challenges due to the natures of datasets, and describe validation strategies to ensure that trained models are not biased.

A Problem Formulation

Let $X \in \mathbb{R}^{n \times d}$ denote the matrix representation of the transformed dataset with n samples and d features and let $y \in \{0, 1\}^n$ denote the corresponding binary labels, where $y = 1$ represents the positive class (fraud or bad credit) and $y = 0$ represents the negative class.

The goal is to learn a function $f: \mathbb{R}^d \rightarrow \mathbb{R}$ that produces a score for each input, which is then converted into a class prediction using a decision threshold t :

$$\hat{y} = \begin{cases} 1, & \text{if } f(x) \geq t \\ 0, & \text{else} \end{cases}$$

The objective is a function f where it can produce the prediction $\hat{y}$ that is accurate as possible for any data, including those which are unseen.

B Models Used

Few models were introduced for fraud detection and credit assessment tasks. Model is a general form of the classification function with parameters which will be obtained by fitting training datasets,

and hyperparameters which will be adjusted during validation phase. Several models are trained and best models are chosen after their performance is compared.

For both tasks, logistic regression model was implemented as a baseline. For the fraud detection task, SVM model and Random Forest model were implemented and for the credit assessment task, Decision Tree, Random Forest, Gradient Boosting, XGBoost, and a Stacked Ensemble meta-learner were implemented in addition to the baseline.

1) *Logistic Regression Model*

Logistic Regression is the most fundamental classification algorithm. Given data $x \in \mathbb{R}^d$, to predict its classification, it first calculates the score using following equation:

$$f(x) = \sigma(w^T x),$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

, where $w \in \mathbb{R}^d$ are learned parameters, and $\sigma(z)$ is the sigmoid function. The output of f is a probability of $y = 1$, which then converted into class prediction $\hat{y}$ with threshold $t = 0.5$.

Due to its simplicity and interpretability, Logistic Regression model serves as a baseline for comparison. In this project, it provides a reference point for evaluating the effectiveness of more complex models.

Logistic Regression model can be called directly using LogisticRegression object from scikit-learn, and no hyperparameters are needed.

2) *Support Vector Machine (SVM) Model*

Support Vector Machine (SVM) model learns the classification by finding a decision boundary that maximizes the margin between two classes. In the non-linear case, the original features are mapped into a higher-dimensional space using a kernel function so that datasets can be separated linearly. In this case, the radial bias function (RBF) is used. The equation for RBF kernel is:

$$K(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right)$$

, where γ controls the influence of individual training points.

The decision function is given by:

$$f(x) = \sum_i (\alpha_i y_i K(x_i, x) + b)$$

SVM model with kernel is effective to high-dimensional feature spaces because it captures complex, non-linear relationships.

SVM model can be called using SVC object from scikit-learn, and the hyperparameter “kernel” is set to “rbf”, so the RBF kernel is applied. There are two hyperparameters: regularization term C , and gamma value γ . C controls the trade-off between maximizing the margin and minimizing classification errors. Low C results in larger margin and simpler decision boundary but will ignore some training points, while high C prioritizes classification of all training points but results in narrower margin and higher risk of overfitting. γ controls influence of single training example. These hyperparameters are experimented during CV training, and best hyperparameters will be chosen.

3) *Decision Tree Model*

A Decision Tree recursively partitions the feature space by selecting at each node the feature and threshold that maximizes information gain. Two impurity criteria are supported. The Gini impurity of a node t is defined as:

$$Gini(t) = 1 - \sum_i p_i^2$$

, where p_i is the weighted fraction of samples of class i at node t , computed using sample weights sw as:

$$p_1 = \frac{\sum sw[y = 1]}{\sum sw}$$

The Shannon entropy of a node is defined as:

$$H(t) = - \sum p_i \log_2(p_i)$$

The information gain of a candidate split dividing n samples into a left partition of size n_L and a right partition of size n_R is:

$$IG = impurity(t) - \frac{n_L}{n} impurity(t_L) - \frac{n_R}{n} impurity(t_R)$$

, where n_L and n_R are the sums of sample weights in each partition rather than raw counts, so that the cost matrix is reflected in split decisions. The tree is built recursively until one of three stopping conditions is met: the current depth reaches *max_depth*, the number of samples at the node falls below $2 \times min_samples_leaf$, or the node contains only one class. Leaf nodes predict the weighted majority class and output a probability:

$$\hat{p} = \frac{\sum sw[y = 1]}{\sum sw[y = 0] + \sum sw[y = 1]}$$

4) *Random Forest Model*

Random Forest is an ensemble method that reduces variance by averaging predictions from many decorrelated decision trees, each trained on a different random bootstrap sample with a random subset of features at each split. Bootstrap sampling draws n samples with replacement from the training set for each tree, so that each tree sees approximately 63% unique samples. At each split, only a randomly selected subset of $\sqrt{d}$ features are considered, which decorrelates the trees. Let $T_b(x)$ denote the probability output of the b -th tree for input x . The final ensemble prediction is the average of all B tree outputs:

$$f(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

Random Forest reduces variance while keeping bias low because individual trees are high-variance low-bias learners, and averaging their uncorrelated predictions cancels out their individual errors without introducing additional bias.

5) *Gradient Boosting Model*

Gradient Boosting builds an ensemble of trees sequentially, where each tree is trained to correct the residuals of the current ensemble. The ensemble is initialized to the logodds of the training labels:

$$F_0 = \log\left(\frac{\bar{p}}{1 - \bar{p}}\right), \bar{p} = \frac{1}{n} \sum_{i=1}^n y_i$$

At each boosting step m , pseudo-residuals are computed as the negative gradient of the log loss with respect to the current ensemble prediction F_{m-1} :

$$r_i = y_i - \sigma(F_{m-1}(x_i)), \sigma(z) = \frac{1}{1 + e^{-z}}$$

A shallow decision tree h_m is fit to the binarized pseudo-residuals. After fitting, the leaf values of h_m are overridden with the weighted mean residual within each leaf j :

$$\gamma_j = \frac{\sum_{i \in \text{leaf } j} sw_i x_i}{\sum_{i \in \text{leaf } j} sw_i}$$

This is the analytically correct leaf update for log loss. The ensemble is then updated as:

$$F_m(x) = F_{m-1}(x) + \eta * h_m(x)$$

, where η is the learning rate controlling the contribution of each tree. The final probability prediction is:

$$\hat{p}(x) = \sigma(F_M(x)) = \frac{1}{1 + e^{-F_M(x)}}$$

Row subsampling with ratio $\rho = 0.8$ is applied at each step, so each tree is fit to a randomly selected 80% of the training samples.

6) *XGBoost Model*

XGBoost (Extreme Gradient Boosting) extends standard Gradient Boosting through second-order Taylor expansion of the loss function and explicit regularization. Given the current ensemble prediction F , the objective for the m -th tree f is approximated as:

$$L^{(m)} \approx \sum_i \left[g_i f(x_i) + \frac{1}{2} h_i f(x_i)^2 \right] + \Omega(f)$$

, where g_i and h_i are the first and second order gradients of the log loss:

$$g_i = p_i - y_i$$

$$h_i = p_i(1 - p_i)$$

To account for the asymmetric cost matrix, gradients of positive class samples ($y_i = 1$) are scaled by the parameter $\omega = \text{scale_pos_weight} = 2$:

$$g_i \leftarrow \omega \cdot g_i, h_i \leftarrow \omega \cdot h_i, \forall i: y_i = 1$$

The regularization term penalizes tree complexity and large leaf weights:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

, where T is the number of leaves, j are leaf weights, γ is the minimum gain threshold, and λ is the L2 regularization coefficient. Minimizing $L(m)$ with respect to j gives the optimal leaf weight:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

, where $G_j = \sum_{i \in \text{leaf}_j} g_i$ and $H_j = \sum_{i \in \text{leaf}_j} h_i$. The gain from a candidate split dividing leaf j into left and right children is:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

A split is only performed if $Gain > 0$. Both row subsampling ($\rho_{row} = 0.8$) and column subsampling ($\rho_{col} = 0.8$) are applied at each tree.

7) *Stacked Ensemble*

The Stacked Ensemble combines the predictions of Random Forest, Gradient Boosting, and XGBoost through a two-layer architecture that learns the optimal weighting between base models. In Layer 1, each base model generates out-of-fold (OOF) probability predictions using 5-fold cross-validation. For fold k , the base model is trained on the remaining $K - 1$ folds and predicts on fold k , producing a meta-feature matrix $Z \in \mathbb{R}^{n \times 3}$:

$$Z_{ij} = P(y_i = 1 | x_i; \text{model}_j \text{ trained on folds } \neq \text{fold}(i))$$

After OOF generation, all base models are retrained on the full training set. In Layer 2, a Logistic Regression meta-learner is trained on Z with sample weights:

$$f_{meta}(Z_i) = \sigma(w^T Z_i + b)$$

, where $w \in \mathbb{R}^3$ are the learned combination weights. The final class prediction is:

$$\hat{y} = \begin{cases} 1, & \text{if } f_{meta}(Z_i) \geq t^* \\ 0, & \text{else} \end{cases}$$

, where t^* is the optimal threshold.

C *Feature Selection*

Feature selection on credit risk dataset was performed using Information Gain (IG) as a filter method, implemented from scratch. Information Gain measures the reduction in uncertainty about the target Y achieved by knowing a feature X :

$$IG(Y, X) = H(Y) - H(Y|X)$$

, where the Shannon entropy of the target is:

$$H(Y) = - \sum_y p(y) \log_2 p(y)$$

, and the conditional entropy is:

$$H(Y|X) = \sum_x p(x) H(Y|X = x)$$

Since all features after scaling are continuous, each feature was discretized into $B = 10$ equal-width bins. For a feature with minimum x_{min} and maximum x_{max} , the bin width is:

$$\Delta = \frac{x_{max} - x_{min}}{B}$$

and sample i is assigned to $\min\left(\left\lceil \frac{x_i - x_{min}}{\Delta} \right\rceil, B - 1\right)$. The conditional entropy is approximated as:

$$H(Y|X) \approx \sum_{b=0}^{B-1} \frac{n_b}{n} H(Y|X \in bin_b)$$

IG was computed on the training set only to prevent leakage of test set label information into the feature selection process. Features with $IG < 0.01$ were discarded. Plot of feature importance via information gain is in Appendix B-4.

D Handling Class Imbalances

Both datasets contain class imbalances, especially for fraud datasets, with only 0.17% of samples are fraud. To not let models learn to classify most samples as negative to increase their accuracy, the Synthetic Minority Over-sampling Technique (SMOTE) was applied to reduce the class imbalance.

SMOTE is a technique for reducing class imbalance by generating entirely new synthetic samples of minority class through linear interpolation. First, one sample from minority class is chosen randomly, and it defines k nearest neighbors also from minority class. Next, one of its nearest neighbors are chosen at random, and draws the vector connecting these two samples. Then, new sample is created at a random point along the connecting vector. Mathematically, new sample is generated by:

$$x_{new} = x_i + \lambda(x_j - x_i)$$

, where x_i and x_j are feature vectors of minority samples chosen randomly, and λ is a random number between 0 and 1. These processes are repeated until enough number of minority samples are created so that class distributions are balanced.

SMOTE can be implemented from the imbalanced-learn library and can be fit to datasets like scikit-learn transformers. When SMOTE was experimented on training dataset, indeed, around 30,000 new samples were generated, and class distribution became 50:50 (See appendix A-7). SMOTE transformer was organized into preprocessing pipeline using pipeline object from imbalanced-learn (not scikit-learn). For our final preprocessing pipeline, first, appropriate column transformations are applied. Then, SMOTE is applied. And finally, classification model is fit (See appendix A-8). Note that SMOTE is only applied to training folds during cross-validation, and is not applied to validation folds nor the test set, to avoid data leakage.

For the German Credit dataset, the class imbalance was addressed through asymmetric cost weighting rather than synthetic oversampling. A cost matrix was defined assigning different misclassification penalties to reflect real financial impact:

$$cost = FN * c_{FN} + FP * c_{FP}$$

where a False Negative (approving a bad loan) carries cost $c_{FN} = 2$, and a False Positive (rejecting a good loan) carries cost $c_{FP} = 1$. The 2:1 ratio was determined experimentally, a 5:1 ratio produced high recall but reduced accuracy below 70%, while 2:1 allowed XGBoost to achieve 75.0% cross-validated accuracy while the Stacked Ensemble maintained the lowest business cost. Cost weights were encoded as `sample_weight` during training. For XGBoost, the equivalent parameter $\omega = scale_pos_weight = 2$ scales the gradients of all positive class samples.

E Validation Strategy

To ensure that models generalize to unseen data, and test for reliability of models, validation is important in model selection process. In this project, cross-validation (CV) was used rather than simple train-validation-test split because CV provides a more reliable and efficient estimate of model performance, especially under limited data and class imbalance.

1) Credit Card Fraud Dataset

Models used for fraud detection tasks are validated by following steps. For each model, first, best hyperparameters are found by CV. Next, models with optimal hyperparameters are evaluated also by CV, performances of each model are compared, and the best model is chosen. Finally, best model is trained with full training dataset, and final performance metric was evaluated using test dataset.

For fraud detection tasks, 5-fold cross validation was used for hyperparameter-tuning and model evaluation. The training dataset is divided into 5 equal subsets, where 4 subsets are used for training, and the remaining subset is used for model evaluation. Training and evaluation processes are repeated 5 times with each subset being used for evaluation for once, and mean metrics on performance on evaluation datasets are reported as average performance and standard deviation of it are reported as stability of the model.

To ensure class distributions are preserved during the CV split, `StratifiedKFold` object from `scikit-learn` was used to implement the 5-folds CV. Indeed, all 5 subsets contain 0.17% of fraud samples. For each fold, standard transformations are adjusted so that only training set will have standard mean and unit variance, and transformations to evaluation set are applied using statistics of training

set. Also, SMOTE is only applied to training datasets, and no new samples are generated on evaluation set.

First, cross validation was performed for few combinations of hyperparameters and based on metrics obtained by CV evaluation process described above, best combinations of hyperparameters are determined. For this project, the hyperparameter combination which produces maximum mean recall is chosen as optimal hyperparameters. Then, another CV is performed on the model with chosen hyperparameters and resulting metric is used for model comparison.

However, due to the large number of datasets, SVM models and RF models were really slow in training. So, reduced dataset was used to train these models. Because of the significant imbalance between classes, reduced samples are generated by keeping all fraud samples and randomly selecting few non-fraud samples. 10,000 non-fraud samples are selected to train SVM model, and 20,000 non-fraud samples are selected to train Random Forest model. This will decrease the training time for these models while still preserving the class imbalance. Same reduced datasets are used for hyperparameter-tuning CV, as training set for model comparison, and as training set for the final model. Also, SVM models were slow even with reduced dataset, so for hyperparameter-tuning process of SVM model, number of cross-validations were reduced to 3-folds.

The final model will be evaluated against test datasets which preserve actual class distributions. Therefore, precision will likely drop significantly. This issue arises because the model was trained on a dataset that was balanced by SMOTE, which shifts the decision boundary towards the minority class, and as a result, the final model will produce many false positives. To address this, decision threshold tuning was introduced, allowing better control over the precision-recall trade-off. Decision thresholds were optimized using a cost-sensitive framework, where each misclassification was assigned unequal costs, and the optimal threshold was selected by minimizing total expected cost over validation predictions:

$$cost = FP * cost(FP) + FN * cost(FN)$$

In this case, false negatives are assumed to be 100 times more expensive than false positives, since missing fraud samples is far more critical.

2) *German Credit Dataset*

For the credit risk assessment task, nested cross-validation was used to provide unbiased performance estimates. The outer loop uses 5-fold stratified cross-validation, preserving the 70:30 class ratio in each fold. For the Stacked Ensemble, an inner 5-fold loop generates OOF meta-features within

each outer training fold, ensuring no information from the evaluation fold enters either the base model training or the meta-learner training.

Decision threshold tuning was performed by sweeping thresholds from 0.1 to 0.9 in steps of 0.01 and selecting the threshold that minimized total business cost:

$$t^* = \operatorname{argmin}_t [FN(t) * 2 + FP(t) * 1]$$

The optimal threshold was found to be $t^* = 0.32$, below the default of 0.5, reflecting the model's conservative probability calibration and the asymmetric cost structure that favors recall over precision.

F Evaluation Metrics

This section describes a method that was used to evaluate model performances. Model performance was evaluated against validation fold during CV to select best hyperparameters and compare models, and against the test set to measure final performance.

1) Credit Card Fraud Dataset

The CV pipeline was implemented so that for each fold: accuracy, precision, recall, F1-score, and ROC-AUC scores are calculated, and mean and standard deviations of these scores are reported. These metrics are calculated and returned for both training datasets and test datasets so they can be compared to check for overfitting. It also returns list of validation samples from all folds, and their corresponding score values output from models. List of validation samples and their score values are used to plot precision-recall curve and for later threshold tuning. Also, confusion matrix was plotted to specifically understand distribution of misclassifications (See appendix A-9 for code snippets for CV evaluation pipeline used for fraud datasets).

Accuracy, precision, recall, and F1-score are calculated as following:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

, where TP , True Positive, is number of positive (fraud) samples that are predicted as positive, TN , True Negative, is number of negative (non-fraud) samples that are predicted as negative, FP , False Positive, is number of negative samples that are predicted as positive, and FN , False Negative, is number of positive samples that are predicted as negative.

Because of the huge imbalance in data, models will predict “non-fraud” for most of the time. Therefore, the accuracy metric is useless. Also, in this case, missing fraud case is far more critical than false alarm, F1-score, the metric which treats false positives and false negatives equally, is meaningless here as well. Since the goal of this project is to accurately detect fraud while minimizing false positives, our objective is to maximize the recall while maintaining reasonable precision. Therefore, recall is compared during hyperparameter-tuning process.

ROC-AUC score measures the area under the ROC curve, which indicates classification accuracy of positive samples as threshold changes. The curves plot true positive rate (TPR) vs. false positive rate (FPR). Mathematically, these are:

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

In other words, the curve measures how much percentage of positive samples will be detected when more percentages of false positives are accepted. Ideally, high true positive rate should be achieved with minimal false positive rate, so the curve should be closer to the top-left corner. The classification is perfect when positive sample detection is perfect with no false positives, which is when the area under the curve becomes 1. However, when there are lot of negative samples, the denominator becomes large, making the false positive rate look small even when number of false positives is large. Therefore, this metric is misleading under heavy class imbalance.

Instead, Precision-Recall (PR) curves are plotted to measure model performance. It shows trade-off between precision and recall. When the classification is perfect, 1.0 precision and recall are achieved at the same time, which is when curve is attached to the top-right corner. Objective of PR curve is similar to ROC curve, but it plots precision and recall, which directly measure performance on the minority class. So, PR curve is more informative evaluation under class imbalance than ROC curve, because precision explicitly penalizes false positives. During CV evaluation, PR curve will be plotted based on validation samples from all folds.

Confusion matrix is another useful way to measure model performance by directly displaying number of TP, TN, FP and FN. During CV evaluation, confusion matrix will be displayed for validation samples from all folds. SVM and RF models are trained from reduced samples so total number of non-fraud samples are different between CV of each model, but all fraud samples are kept in reduced dataset. Therefore, lower half of confusion matrix can be interpreted and compared directly even when models are evaluated using reduced datasets.

2) *German Credit Dataset*

Model performance was evaluated using the following metrics, all computed manually from the confusion matrix entries TP, TN, FP, FN.

The AUC was computed manually by sweeping 200 threshold values, computing the True Positive Rate (TPR) and False Positive Rate (FPR) at each threshold:

$$TPR(t) = \frac{TP(t)}{TP(t) + FN(t)}$$

$$FPR(t) = \frac{FP(t)}{FP(t) + TN(t)}$$

The (FPR, TPR) pairs were sorted by FPR and the area computed via the trapezoidal rule:

$$AUC = \sum_{k=1}^K \frac{1}{2} (TPR_k + TPR_{k-1}) (FPR_k + FPR_{k-1})$$

Business cost is the primary evaluation metric. Type I error is defined as the False Positive Rate $\frac{FP}{FP + TN}$, representing good applicants incorrectly rejected. Type II error is defined as the False Negative Rate $\frac{FN}{FN + TP}$, representing bad applicants incorrectly approved. Under the adopted cost matrix, Type II errors are twice as costly as Type I errors.

IV RESULTS

For both fraud detection and credit assessment tasks, hyperparameter-tuned models are compared using validation and evaluation techniques described above. Based on the comparison, best models are chosen for each task, and final evaluation is made by measuring final models' performance on the test dataset.

A *Credit Card Fraud Dataset*

For fraud detection task, first, Logistic Regression model was implemented as a baseline model. Since there are no hyperparameters for this model, it was evaluated against all training datasets by

cross-validation. According to the validation result shown in Table A-10-1, the Logistic Regression model achieved mean recall of 0.904 with standard deviation of 0.023 and mean precision of 0.057 with standard deviation of 0.002. During the entire CV process, 38 out of 394 fraud samples were missed, according to the confusion matrix in Figure A-10-2. Also, according to the precision-recall curve plotted in Figure A-10-3, in order to accurately detect fraud samples using this model, lot of false positives must be made.

Table A-10-1: Metrics table for Logistic Regression model.

	training mean	validation mean	validation std
accuracy	0.974	0.974	0.002
precision	0.058	0.057	0.002
recall	0.928	0.904	0.023
f1	0.110	0.107	0.004
roc_auc	0.951	0.939	0.011

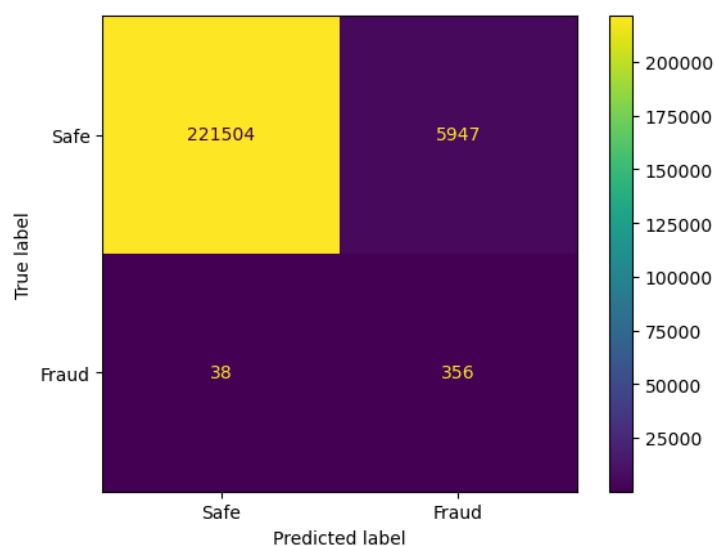


Figure A-10-2: Confusion matrix for Logistic Regression model.

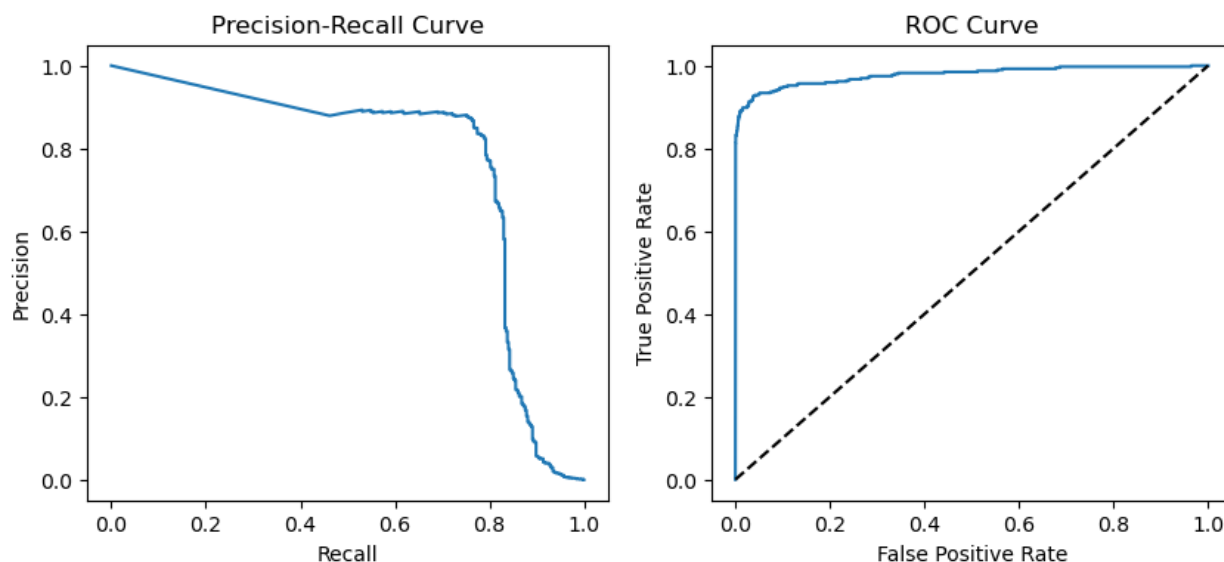


Figure A-10-3: Precision-recall and ROC curves for Logistic Regression model.

Next, SVM model with RBF kernel was implemented. Due to the slow training time, this model was trained with reduced dataset as described in section III. After the hyperparameter-tuning 3-fold CV process, the optimal hyperparameters are determined to be $C = 0.1$ and $\gamma = 0.1$. Then, the SVM model with these hyperparameters are evaluated by 5-fold CV using same reduced dataset. According to the CV result shown in Table A-11, SVM model achieved mean recall of 0.929 with standard deviation of 0.02, and mean precision of 0.386 with standard deviation of 0.024. Although there was only slight improvement in average recall, there was a huge improvement in precision compared to the baseline.

Table A-11: Metrics table for SVM model.

	training mean	validation mean	validation std
accuracy	0.958	0.941	0.005
precision	0.477	0.386	0.024
recall	0.987	0.929	0.020
f1	0.643	0.545	0.026
roc_auc	0.972	0.935	0.012

After that, Random Forest model was implemented also with reduced dataset, but different from that used when training SVM model, with 20,000 non-fraud samples as described in section III. But the training time was relatively faster than SVM, hyperparameter-tuning was done with 5-fold CV.

Best hyperparameters were determined so that Random Forest model will contain 50 trees, each with maximum depth of 20, and minimum of 10 samples are contained in the deepest layer. Then, the Random Forest model is also evaluated by 5-fold CV using same reduced dataset. According to the CV result shown in table A-12, Random Forest model achieved mean recall of 0.866 with standard deviation of 0.02, and mean precision of 0.859 with standard deviation of 0.022. Therefore, Random Forest model had significant improvement on precision. However, recall has dropped. Also, some metrics were not stable across folds with higher standard deviations compared to two previous models.

Table A-12: Metrics table for Random Forest model.

	training mean	validation mean	validation std
accuracy	0.999	0.995	0.001
precision	0.939	0.859	0.022
recall	0.999	0.866	0.020
f1	0.968	0.862	0.019
roc_auc	0.999	0.931	0.010

In addition to direct metric comparisons, feature importance plot was generated for Random Forest model. Feature importance plot simply shows how much each feature contributes to the final decision making. According to the feature importance plot in Figure A-13, a small subset of latent features, such as V15, V13 and V5 have shown high importance according to the importance plot. This means that PCA has extracted few latent features which make big contributions in the fraud detection. However, non-latent features, time and amount, had little importance, suggesting that fraud is not really determined by these features.

After metrics of all three models are computed, they are compared for final model selection. First, mean CV metrics of them are compared in table A-14. When comparing recall of each model, SVM has highest recall of 0.929, while Random Forest has lowest recall of 0.866. When comparing precisions, Random Forest has highest precision of 0.859, Logistic Regression has poor precision of 0.057, while SVM has low, but relatively stable precision compared to Logistic Regression model. Therefore, Logistic Regression accurately detects fraud, but has way too many false positives, Random Forest model is very precise but misses fraud, and SVM model is best in detecting fraud while being most precise in its prediction.

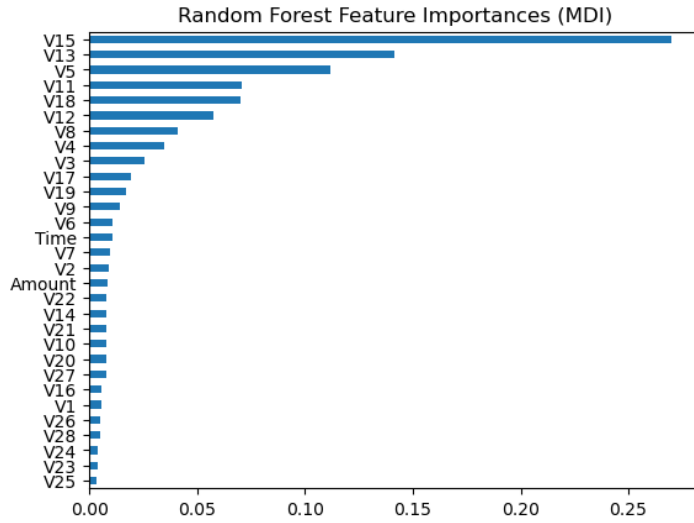


Figure A-13: Feature importance plot.

Table A-14: Mean metrics of three models during CV evaluation.

	accuracy	precision	recall	f1	roc_auc
Logistic Regression	0.974	0.057	0.904	0.107	0.939
SVM	0.941	0.386	0.929	0.545	0.935
Random Forest	0.995	0.859	0.866	0.862	0.931

Confusion matrices and precision-recall curves for classification of total validation sets for all three models are also displayed in Figure A-15 and A-16, respectively. Since number of fraud samples are same for all training datasets, FN and TP values can be compared directly. As shown in matrices, Random Forest model makes most misclassification on fraud samples while SVM model most accurately detects fraud samples even when trained with reduced dataset. According to the precision-recall curves, to accurately detect fraud transactions using Logistic Regression model, it must make lot of false positives. While for other two models, they seem to be able to maintain relatively high precision for high recall. Random Forest seems to have best balance in classification of two classes, but only slight difference in precision trade-off for high recall between Random Forest and SVM models.

Standard deviations of recall across cross-validation folds are compared to determine stability of models. According to Table A-17, all models seem to have stable performance with relative standard deviation of around 0.023.

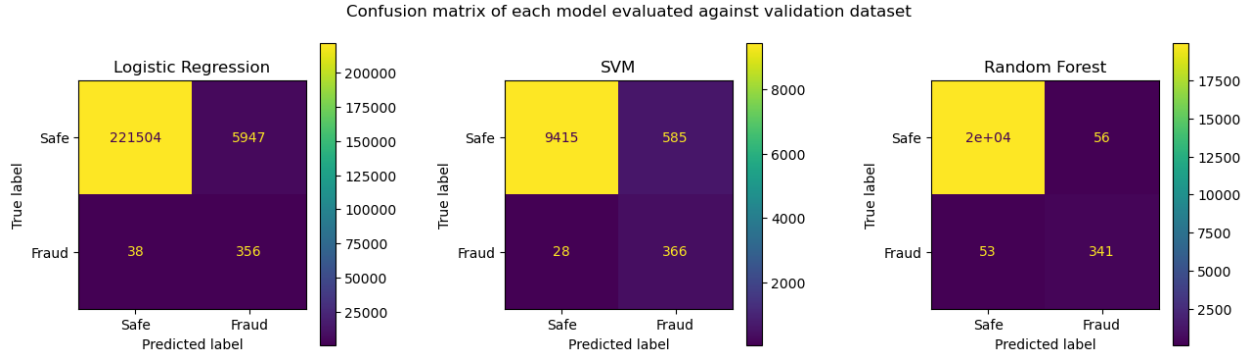


Figure A-15: Confusion matrices for classification of total validation sets for all three models.

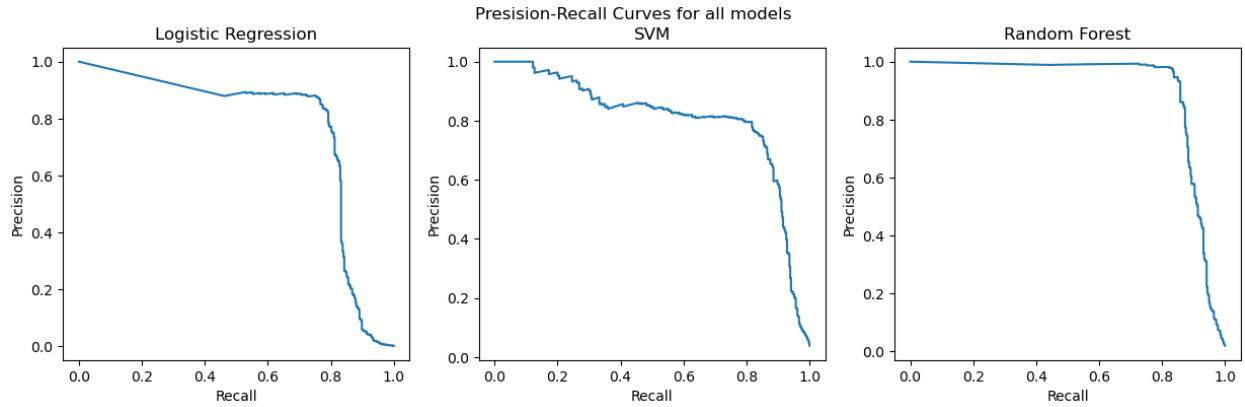


Figure A-16: PR curve for classification of total validation sets for all three models.

Table A-17: Tables of mean and variance of CV recall scores for each model.

	Mean	Standard Deviation	Relative Std
Logistic Regression	0.904	0.023	0.025
SVM	0.929	0.020	0.022
Random Forest	0.866	0.020	0.023

Finally, comparison of training recall and validation recall across cross-validation folds are compared in Table A-18 to check for overfitting. According to the table, Random Forest has almost perfectly detected fraud samples for training sets but significantly lower score for validation sets. This shows large gap between training and validation scores, indicates overfitting in the model. In contrast, other two models demonstrated better generalization with smaller gap between performance on training and validation dataset, with high recall score for validation dataset on SVM model.

Table A-18: Tables of comparison of training recall and validation recall across cross-validation folds.

	Training Recall	Validation Recall
Logistic Regression	0.928	0.904
SVM	0.987	0.929
Random Forest	0.999	0.866

Among three models, SVM model was concluded to be the most effective in detecting fraud in transactions among three models, with achieving highest recall while maintaining moderate false positive rates. In this context, correctly detecting fraud transactions is far more important than incorrectly classifying safe transactions as fraud. So, although Random Forest model achieved highest accuracy, precision, and F1-scores, it was decided to be not the best choice for fraud detection tasks because of its low recall and evidence of overfitting. In contrast, SVM model demonstrated its ability to accurately detect fraud detection while being stable in its performance, making it best model for fraud detection.

After best model was decided to be SVM model, its decision threshold was tuned using cost-sensitive framework described in section III with total validation sets from CV framework. However, this framework resulted in an optimal boundary of -0.08, which is close to the default decision boundary, 0, as shown in Figure A-19. As a result, the original SVM was already tuned for optimal threshold for minimizing cost, and lower precision score is expected for final model.

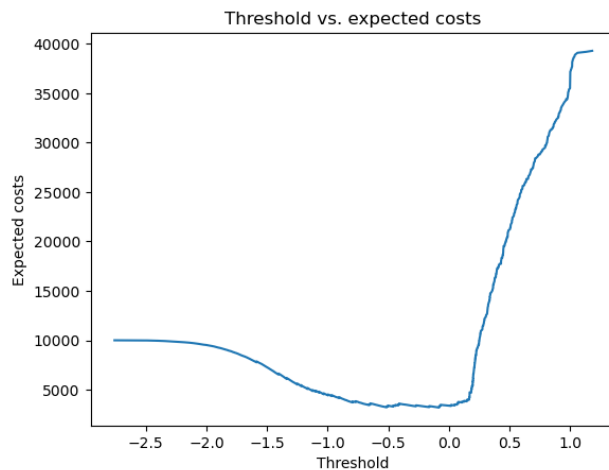


Figure A-19: Plots of expected costs vs. threshold.

SVM model was trained with same reduced dataset with optimal hyperparameters and is deployed as a final model. Then, full test datasets are used for final model evaluation. For the final result, the model was able to detect fraud transactions with recall score of 0.926 and precision score of 0.023.

According to the final confusion matrix in Figure A-20, 3,887 out of 56,864 non-fraud samples are incorrectly classified as fraud, and 7 out of 98 fraud transactions are missed.

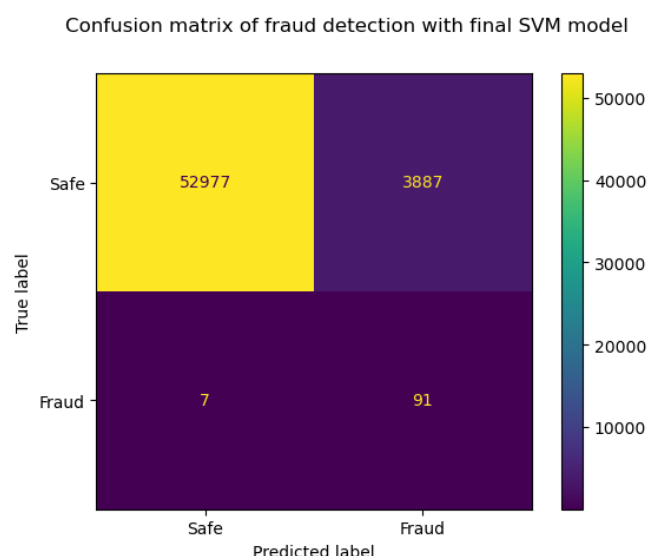


Figure A-20: Final confusion matrix.

B German Credit Dataset

All five models were trained on the 800-sample training set with 2:1 cost weights and evaluated on the held-out 200-sample test set. Cross-validation results are reported as mean $\pm$ standard deviation over 5 folds.

XGBoost achieved the highest cross-validated accuracy at 0.750, meeting the project target of greater than 75% accuracy. The Stacked Ensemble achieved the lowest business cost (94.0) and highest recall (0.879) across all cross-validation folds, demonstrating ensemble superiority over all individual base models on the primary business metric. The Stacked Ensemble also achieved the lowest Type II error (0.083) on the test set, approving only 5 bad loans out of 60. Random Forest achieved the highest AUC (0.787), indicating the strongest overall discriminative ability across all decision thresholds.

The credit scoring results reveal several important insights about model behavior under asymmetric cost constraints and the effectiveness of ensemble methods.

The choice of cost weight significantly impacts the accuracy-recall tradeoff. At a 5:1 weight ratio, all models exhibited recall above 0.87 but cross-validated accuracy below 65%. Reducing the ratio to

2:1 allowed XGBoost to reach 75.0% cross-validated accuracy while the Stacked Ensemble maintained the lowest business cost. This demonstrates that cost weight selection is a critical hyperparameter that must be tuned experimentally rather than set based on domain knowledge alone.

Checking account status dominated the Information Gain feature ranking with a score of 0.079, more than twice the score of the next feature, Duration (0.039). The presence and balance level of a checking account appears to be a more reliable predictor of credit risk than loan characteristics, consistent with the notion that account behavior reflects broader financial discipline.

The Stacked Ensemble demonstrates that combining model predictions through a learned meta-learner with cost-sensitive weighting outperforms any single model on both business cost and recall. The meta-learner assigned the highest weight to XGBoost ($w_{XGB} = 1.68$) and Random Forest ($w_{XGB} = 1.37$), consistent with their individual performance rankings. For regulatory contexts requiring high accuracy, XGBoost at the default threshold of 0.5 is recommended, achieving 75.0% cross-validated accuracy with a competitive business cost of 102.4. For pure cost minimization, the Stacked Ensemble at threshold $t^* = 0.32$ is recommended with a cross-validated cost of 94.0 and recall of 0.879.

The XGBoost second-order gradient formulation produced demonstrably lower training loss curves compared to standard Gradient Boosting, consistent with the theoretical advantage of using curvature information for split quality assessment. The manual Information Gain implementation using 10-bin discretization produced feature rankings consistent with exploratory data analysis findings, validating the approximation despite its discrete treatment of continuous features.

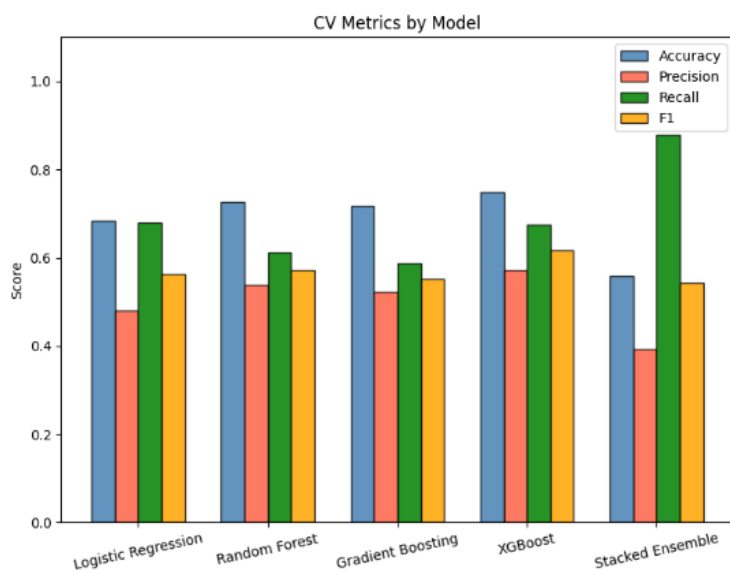


Figure B-22: CV Metrics by Model

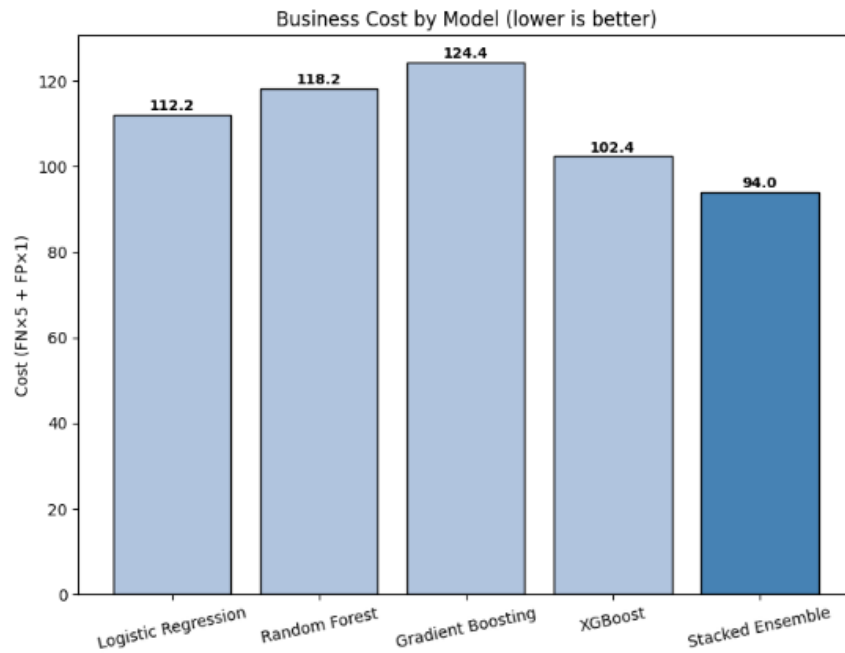


Figure B-23: Business Cost by Model

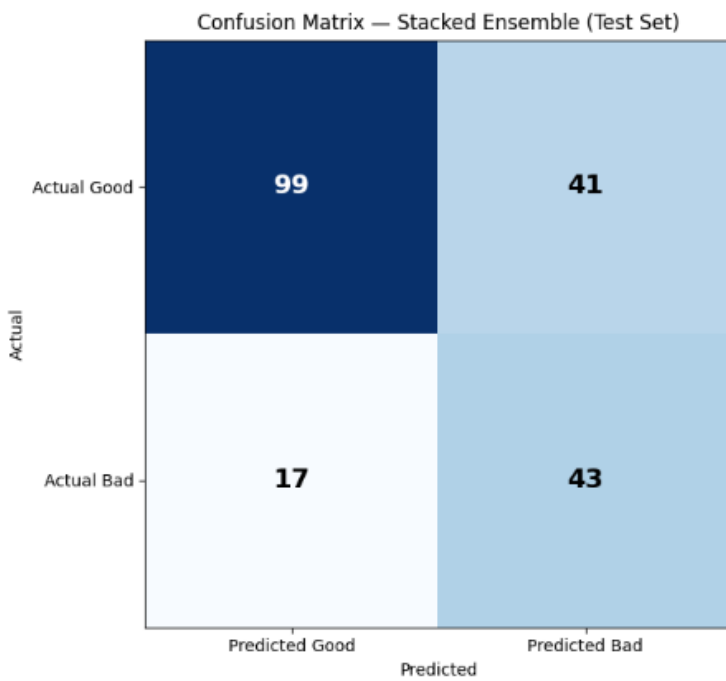


Figure B-24: Confusion Matrix for Stacked Ensemble



Figure B-25: Recall vs Business Cost Comparison for all models

V DISCUSSIONS

The results demonstrate that model performance varies significantly across algorithms and evaluation settings. For fraud detection tasks, the SVM achieved high recall but suffered from low precision when evaluated on the test set. While Random Forest provided a more balanced trade-off between precision and recall. For SVM results, while cross-validation results showed moderate precision, significant drop in precision was observed, while maintaining high recall. This behavior has occurred because of new fraud samples generated by SMOTE, and use of reduced dataset while training. These changed the class distribution and shifted the decision boundary toward the minority class. Therefore, when evaluated on the test set with true class distribution, the model over-predicted the positive class, resulting in many false positives.

Decision threshold tuning was implemented to address the imbalance between precision and recall. However, cost-based threshold optimization resulted only slight change from the default decision boundary. This means that cost sensitivity was already incorporated in the SVM model. This suggests that imbalance techniques such as SMOTE can significantly influence the effective decision boundary, reducing the impact of threshold adjustment after training.

Comparing two complex models used in fraud detection tasks, Random Forest achieved higher precision without significant loss in recall, indicating great robustness to class imbalance. This is likely due to its ensemble structure and ability of Random Forest model to model complex interactions

without defining strict decision thresholds. In contrast, SVM was more sensitive to the training distribution with strict definition of decision boundary, resulting in over-prediction of the minority class.

This study has several limitations. First and obviously, due to computational constraints, SVM and Random Forest hyperparameter tuning and model selection were performed on a reduced dataset, which may limit model performance on the full dataset. Second, while SMOTE improves minority class representation, it may introduce unknown patterns that do not exist in real-world data.

One of the important aspects that was imported during the fraud detection task is to align model evaluation with real-world objectives. For example, prioritizing recall is critical, since not detecting fraud transactions could result in greater loss, but too much false positives may decrease the trust of the model. This means that cost-sensitive threshold selection is essential for deployment.

Possible future work for fraud detection tasks includes exploring more advanced techniques to handle imbalanced data, exploring more machine learning models to find one which achieves high recall while maintaining relatively high precision also for full dataset, and exploring ways to implement threshold selection during cross-validation training.

For the credit scoring task, results demonstrate that ensemble methods outperform individual models on the primary business metric. The Stacked Ensemble achieved the lowest cross-validated business cost of 94.0 and the highest recall of 0.879, while XGBoost achieved the highest cross-validated accuracy of 75.0%, meeting the project target. A key finding was that the choice of cost weight significantly influences the accuracy-recall tradeoff. At a 5:1 cost ratio, models achieved high recall but cross-validated accuracy fell below 65%, while reducing the ratio to 2:1 improved accuracy to 75.0% while maintaining competitive recall. This suggests that cost weight selection must be treated as a hyperparameter and tuned experimentally rather than set by domain knowledge alone.

Feature selection via Information Gain revealed that checking account status was the single most predictive attribute, with a score of 0.079, more than twice the score of the next feature, loan duration (0.039). This indicates that financial account behavior is a stronger predictor of credit risk than loan characteristics themselves, which is consistent with domain knowledge that account management reflects broader financial discipline.

Similar to the fraud detection task, decision threshold tuning proved effective for the credit scoring task. Lowering the Stacked Ensemble threshold from 0.5 to 0.32 reduced test set business cost from 126 to 103, an 18% improvement, while increasing recall from 0.717 to 0.917. This reinforces

the finding that threshold selection is essential when misclassification costs are asymmetric, and that the default threshold of 0.5 is rarely optimal in cost-sensitive financial applications.

This study has several limitations for the credit scoring task. The German Credit dataset contains only 1,000 samples, which limits the statistical reliability of nested cross-validation estimates, as evidenced by the relatively high standard deviations observed across folds. Additionally, all tree-based models were implemented from scratch in pure Python without vectorization, resulting in significantly longer training times compared to optimized library implementations. This constrained the number of estimators and depth that could be practically explored.

Possible future work for the credit scoring task includes exploring SMOTE or ADASYN as an alternative or complement to cost weighting for handling class imbalance, applying hyperparameter tuning via grid search or Bayesian optimization to further improve model performance, and investigating SHAP values for feature importance analysis to provide more interpretable explanations suitable for regulatory compliance.

VI CONCLUSIONS

This project aimed to develop and evaluate machine learning models for two financial tasks: credit card fraud detection and credit risk assessment, where both tasks had class imbalance on datasets, using multiple models such as Logistic Regression, SVM, Random Forest, Decision Tree, Gradient Boosting, XGBoost, and a Stacked Ensemble, evaluated using cross-validation and cost-sensitive metrics.

For fraud detection tasks, the SVM model achieved high recall but suffered from low precision on the imbalanced test set, while Random Forest provided a more balanced performance. In the credit scoring task, XGBoost achieved the highest cross-validated accuracy of 75.0%, and the Stacked Ensemble demonstrated superiority over all individual models by achieving the lowest business cost of 94.0 and the highest recall of 0.879 across cross-validation folds. Decision threshold tuning further reduced test set business cost by 18%, highlighting the practical value of cost-sensitive threshold selection.

A key insight is that handling class imbalance during training significantly affects model behavior by shifting decision boundary toward the minority class. Also, proper validation techniques such as cross-validation were critical to obtaining reliable performance estimates. Furthermore, cost weight selection and decision threshold tuning are essential hyperparameters in financial classification

tasks where misclassification costs are asymmetric, and must be tuned experimentally rather than set by assumption.

This study highlights the importance of proper validation and careful handling of class imbalance in machine learning application in finance tasks. These considerations are essential for developing models that perform reliably in real-world settings. Future work may include continuing exploration of models to accurately detect issues without too much false positives and extending the analysis to larger and more diverse datasets. Development of such models will significantly decrease global financial loss and significantly improve financial stability.

REFERENCES

- [1] R. V., et al. (2025). Credit Card Fraud Data Analysis and Prediction Using Machine Learning Algorithms. *Security and Privacy*, Wiley. <https://onlinelibrary.wiley.com/doi/10.1002/spy2.70043>
- [2] Nwafor, C.N., Nwafor, O., & Brahma, S. (2024). A machine learning-based credit risk prediction engine system using a stacked classifier and a filter-based feature selection method. *Journal of Big Data*, Springer Nature. <https://link.springer.com/article/10.1186/s40537-024-00882-0>
- [3] Chawla, N.V., et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
- [4] He, H., et al. (2008). ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning. *IEEE International Joint Conference on Neural Networks*.

VITA

Akhil Gotkhindi is a graduate student studying Robotics with concentration in Mechanical at Northeastern University, with an expected graduation date of 2026. He obtained his undergraduate degree in Mechanical Engineering, at R V College of Engineering, Bengaluru, India. His academic interests include wearable robotics, exoskeleton control systems, sensor-based state estimation, and reinforcement learning. He has experience designing and validating a quasi-direct-drive ankle exoskeleton with real-time gait phase detection using IMU-based finite state machine control, as well as implementing and comparing deep reinforcement learning agents across Atari environments. He is interested in applying robotics, control systems, and machine learning concepts to real-world problems such as assistive and rehabilitation technology. He also brings industry experience in vehicle dynamics and braking systems from his work at Hero MotoCorp and ESTAT Actuators, where he contributed to CBS and ABS development, suspension tuning, and test automation.

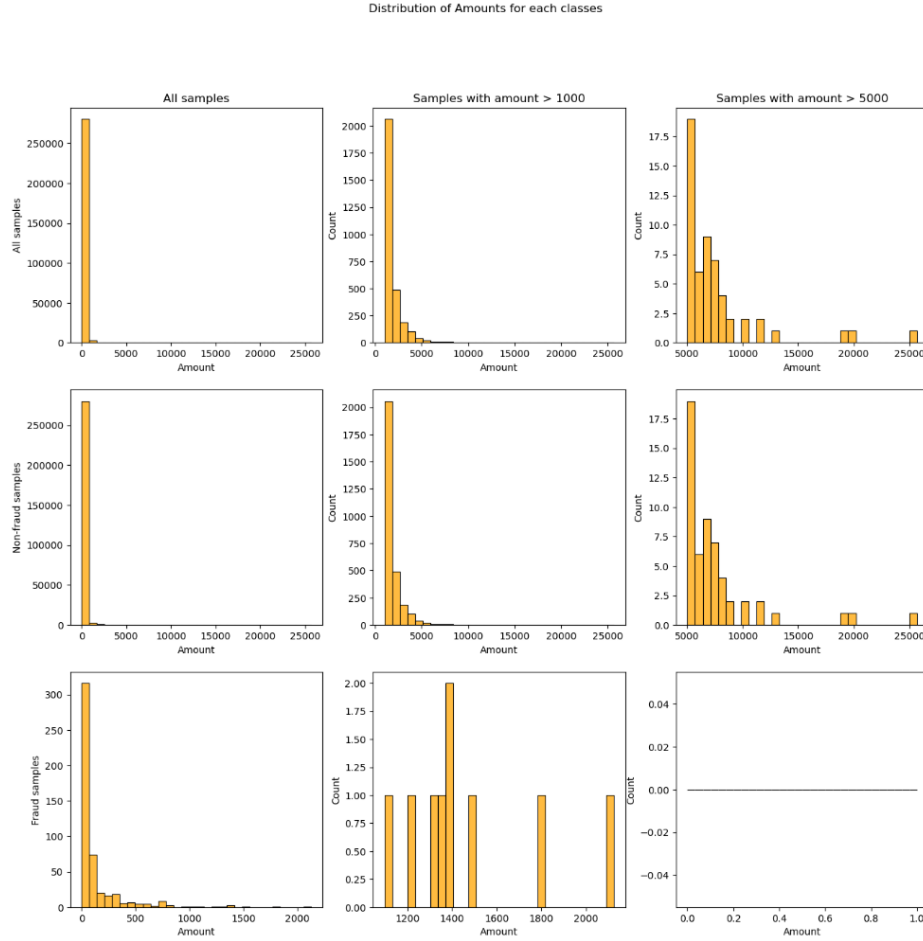
Masahiro Uchida is a graduate student studying Artificial Intelligence with concentration in Machine Learning at Northeastern University, Khoury College of Computer Science, with an expected graduation date of 2027. He obtained his undergraduate degree in Computer Science and Mathematics, also at Northeastern University, Khoury College of Computer Science in 2025. His academic interests include machine learning, mathematics, and programming. He has experience working with statistical modeling and developing machine learning algorithms for classification tasks. He is interested in applying machine learning, computer science, and mathematical concepts to real-world problems such as finance.

APPENDIX

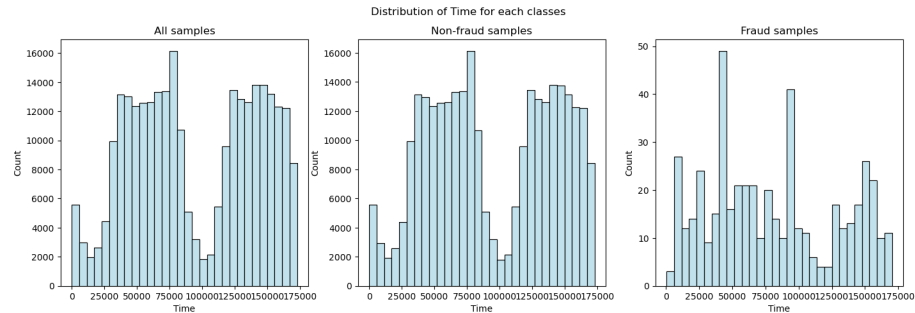
Appendix A

This section contains figures and code snippets that are related to credit card fraud detection tasks.

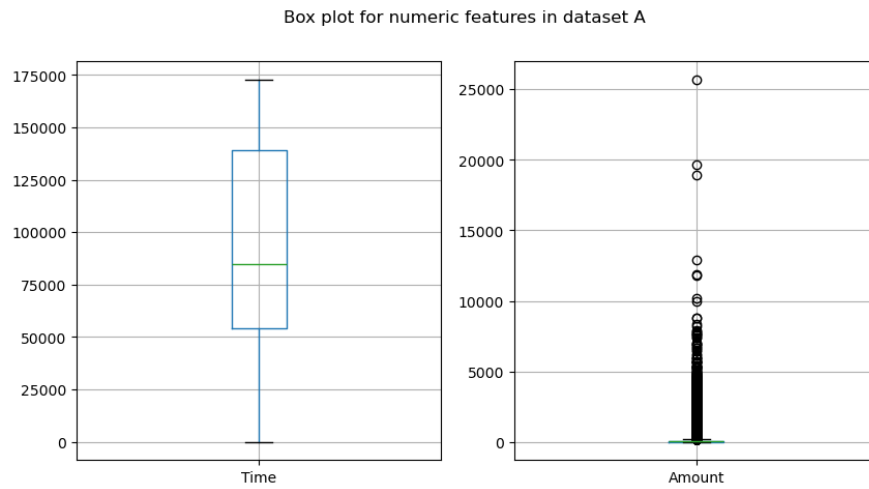
A-1: Histogram of feature distributions for “Amount” feature in fraud dataset.



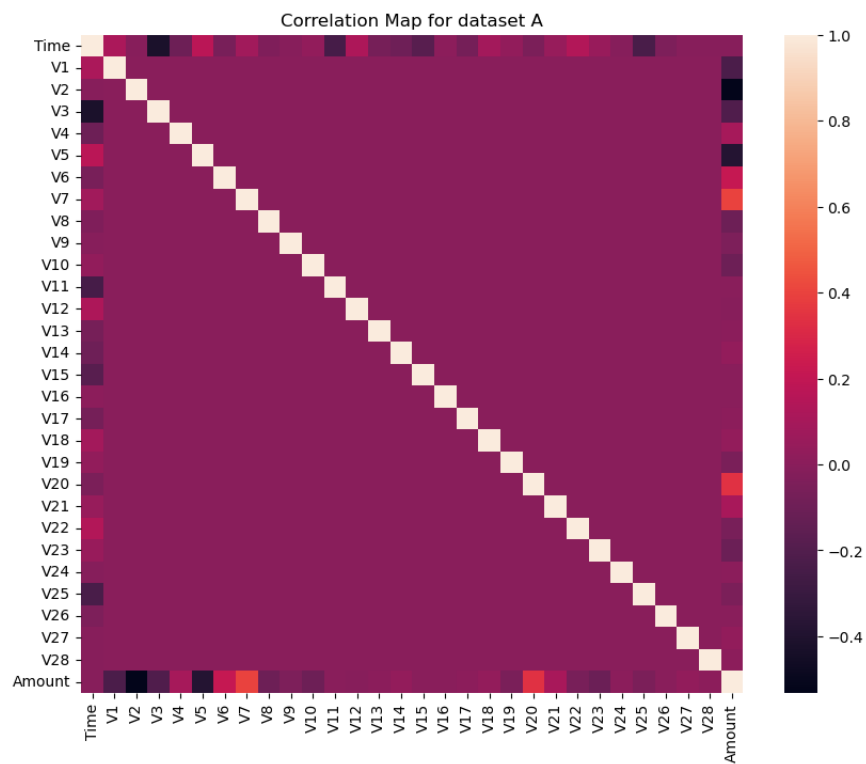
A-2: Histogram of feature distributions for “Time” feature in fraud dataset.



A-3: Boxplot of “Time” and “Amount” features in fraud dataset.



A-4: Correlation map for all features in fraud dataset.



A-5: Code for transformation pipeline for fraud dataset.

```
def log_transform(X):
    """
    Apply log(1 + x) for column with significant skews
    """
    return np.log1p(X)
```

```

log_transformer_pipeline_a = sk_Pipeline(steps=[
    ("log", FunctionTransformer(log_transform)),
    ("std", StandardScaler())
])

# Log transformer for "Amount" to handle right skew
# StandardScaler to "Amount" and "Time"
# Remain PCA components untouched
processor_a = ColumnTransformer(
    transformers=[
        ("log", log_transformer_pipeline_a, ["Amount"]),
        ("std", StandardScaler(), ["Time"]),
    ],
    remainder="passthrough"
)

```

```

Xa_processed = processor_a.fit_transform(Xa)

```

A-6: Code for train-test split for fraud dataset, and confirmation that class distributions are preserved.

```

from sklearn.model_selection import train_test_split

Xa_train, Xa_test, ya_train, ya_test = train_test_split(Xa, ya, test_size=0.2, stratify=ya, random_state=42)

# Check the class distribution for training and testing data
print("Length of A =", len(ya))
print("Length of A Train =", len(ya_train))
print("Length of A Test =", len(ya_test))
print()
print("Distribution for A =", np.mean(ya))
print("Distribution for A Train =", np.mean(ya_train))
print("Distribution for A Test =", np.mean(ya_test))

Length of A = 284807
Length of A Train = 227845
Length of A Test = 56962

Distribution for A = 0.001727485630620034
Distribution for A Train = 0.001729245759178389
Distribution for A Test = 0.0017204452090867595

```

A-7: Code to test SMOTE transformation on training dataset and confirmation that class distributions became 50:50.

```

print("Testing that SMOTE will generate samples so that class distributions will be balanced")
print("Before SMOTE:")
print("Length of A Train =", len(ya))
print("Distribution for A Train =", np.mean(ya))
print()

Xa_smote, ya_smote = SMOTE().fit_resample(Xa_processed, ya)

print("After SMOTE:")
print("Length of A Train =", len(ya_smote))
print("Distribution for A Train =", np.mean(ya_smote))

Testing that SMOTE will generate samples so that class distributions will be balanced
Before SMOTE:
Length of A Train = 284807
Distribution for A Train = 0.001727485630620034

After SMOTE:
Length of A Train = 568630
Distribution for A Train = 0.5

```

A-8: Code for preprocessing pipeline for fraud dataset.

```

from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as imb_Pipeline

def classifier_pipeline_a(model):
    """
    Get classification pipeline for dataset A with given model
    """
    pipeline = imb_Pipeline([
        ("transform", processor_a),
        ("smote", SMOTE(random_state=42)),
        ("classification", model)
    ])

    return pipeline

```

A-9: code snippets for CV evaluation pipeline used for fraud dataset.

```

def get_metrics(y_true, y_pred):
    """
    Compute accuracy, precision, recall, f1, and roc_auc for given classification.
    """
    list_metric_names = ["accuracy", "precision", "recall", "f1", "roc_auc"]
    list_metrics = [accuracy_score, precision_score, recall_score, f1_score, roc_auc_score]
    dict_metrics = dict()

    for metric_name, metric in zip(list_metric_names, list_metrics):
        score = metric(y_true, y_pred)
        dict_metrics[metric_name] = score

    return dict_metrics

```

```
def evaluate(model, X, y, score="probability"):
    """
    Evaluate performance on the model by predicting X and compare it to y.
    Return "accuracy", "precision", "recall", "f1" and "roc_auc" by dictionary.
    Also return predictions and scores.
    """
    y_pred = model.predict(X)

    if score == "probability":
        y_score = model.predict_proba(X)[:, 1]
    elif score == "decision":
        y_score = model.decision_function(X)

    dict_this_metrics = get_metrics(y, y_pred)

    return dict_this_metrics, y_pred, y_score
```

```
def evaluate_cv(pipeline, X, y, cv, score="probability", print_itr=False):
    """
    Measure performance of the pipeline by Cross-Validation and
    report its "accuracy", "precision", "recall", "f1" and "roc_auc".
    Score is measured using "predict_proba", but will use "decision_function" if score="decision".
    Also return all y validation set and probability prediction set for precision_recall_curve.
    Also return confusion matrix of all validations during cv.
    """
    list_metrics = ["accuracy", "precision", "recall", "f1", "roc_auc"]
    list_y_val = list()
    list_probs = list()
    list_y_pred = list()
    dict_metrics = dict()
    dict_training = dict()
    fold = 0

    for metric in list_metrics:
        dict_metrics[metric] = list()
        dict_training[metric] = list()

    for train_idx, val_idx in cv.split(X, y):
        if print_itr:
            print("Fold", fold)
            fold += 1

        X_train = X.iloc[train_idx]
        y_train = y.iloc[train_idx]
        X_val = X.iloc[val_idx]
        y_val = y.iloc[val_idx]

        pipeline.fit(X_train, y_train)
        dict_this_metrics, y_pred, probs = evaluate(pipeline, X_val, y_val, score)
        dict_this_training, _, _ = evaluate(pipeline, X_train, y_train, score)

        for metric in list_metrics:
            dict_metrics[metric].append(dict_this_metrics[metric])
            dict_training[metric].append(dict_this_training[metric])
```



```

list_y_val.extend(y_val)
list_probs.extend(probs)
list_y_pred.extend(y_pred)

for metric in list_metrics:
    scores = dict_metrics[metric]
    training_score = np.mean(dict_training[metric]).item()
    mean_score = np.mean(scores).item()
    std_score = np.std(scores).item()
    dict_metrics[metric] = np.round(np.array([training_score, mean_score, std_score]), 3)

df_metric = pd.DataFrame(dict_metrics).transpose()
df_metric.columns = ["training mean", "validation mean", "validation std"]
cm = confusion_matrix(list_y_val, list_y_pred)

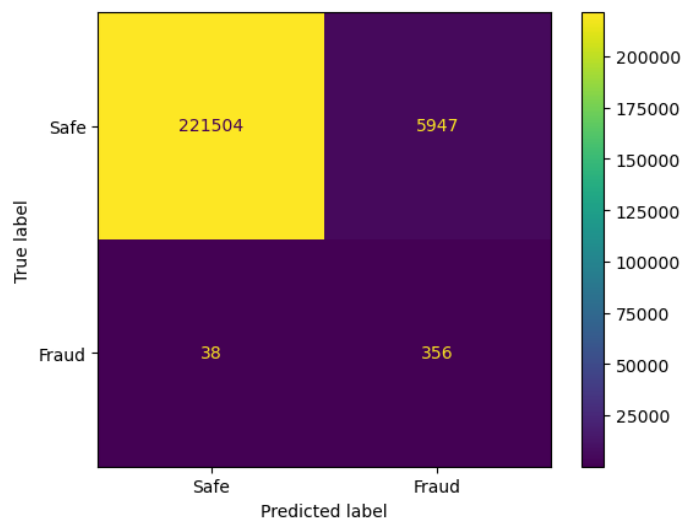
return df_metric, list_y_val, list_probs, cm

```

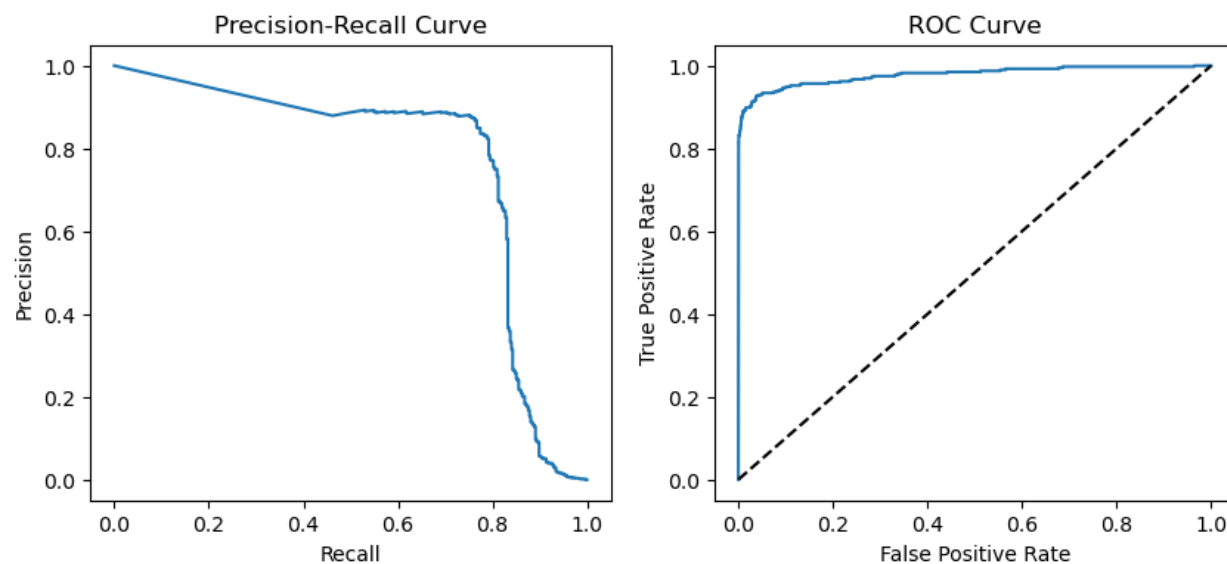
A-10-1: Metrics table for Logistic Regression model.

	training mean	validation mean	validation std
accuracy	0.974	0.974	0.002
precision	0.058	0.057	0.002
recall	0.928	0.904	0.023
f1	0.110	0.107	0.004
roc_auc	0.951	0.939	0.011

A-10-2: Confusion matrix for Logistic Regression model.



A-10-3: Precision-recall and ROC curves for Logistic Regression model.



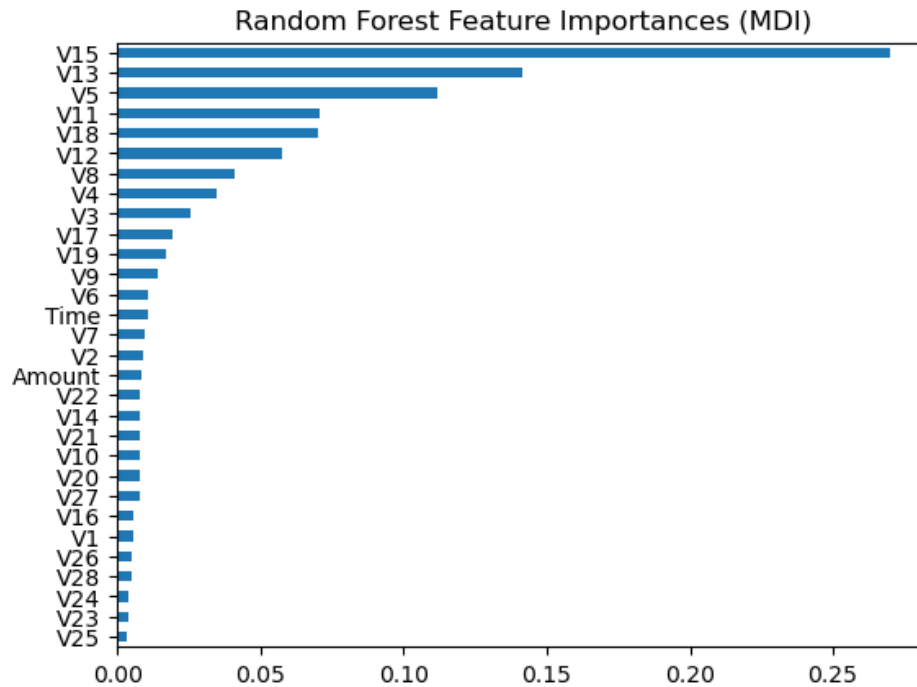
A-11: Metrics table for SVM model.

	training mean	validation mean	validation std
accuracy	0.958	0.941	0.005
precision	0.477	0.386	0.024
recall	0.987	0.929	0.020
f1	0.643	0.545	0.026
roc_auc	0.972	0.935	0.012

A-12: Metrics table for Random Forest model.

	training mean	validation mean	validation std
accuracy	0.999	0.995	0.001
precision	0.939	0.859	0.022
recall	0.999	0.866	0.020
f1	0.968	0.862	0.019
roc_auc	0.999	0.931	0.010

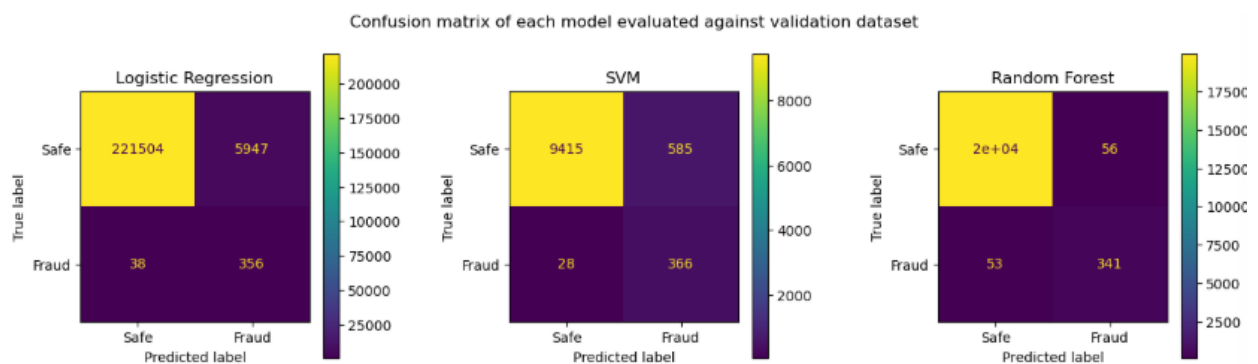
A-13: Feature importance plot.



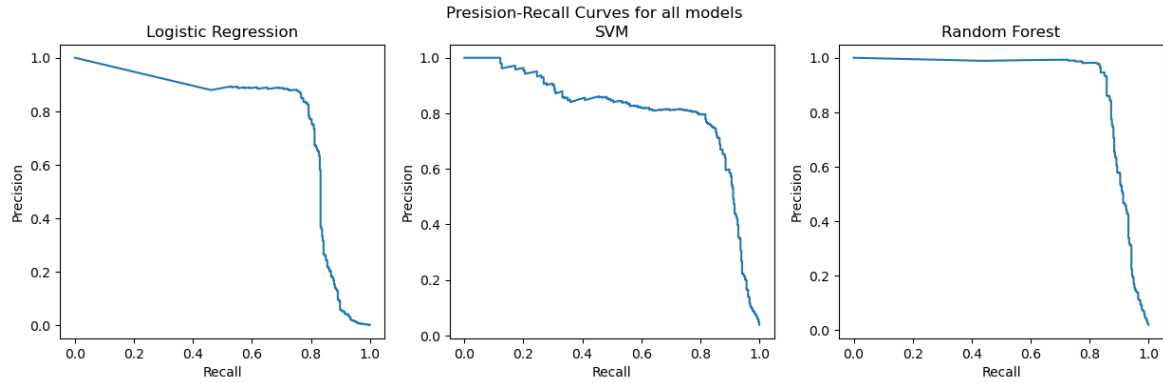
A-14: Mean metrics of three models during CV evaluation.

	accuracy	precision	recall	f1	roc_auc
Logistic Regression	0.974	0.057	0.904	0.107	0.939
SVM	0.941	0.386	0.929	0.545	0.935
Random Forest	0.995	0.859	0.866	0.862	0.931

A-15: Confusion matrices for classification of total validation sets for all three models.



A-16: PR curve for classification of total validation sets for all three models.



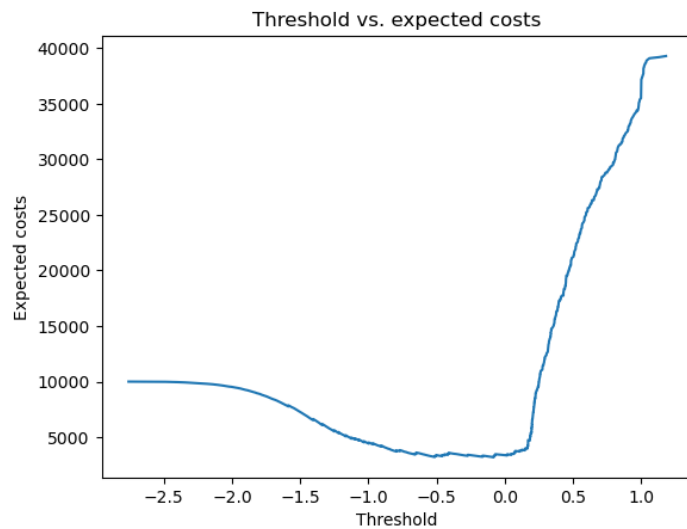
A-17: Tables of mean and variance of CV recall scores for each models.

	Mean	Standard Deviation	Relative Std
Logistic Regression	0.904	0.023	0.025
SVM	0.929	0.020	0.022
Random Forest	0.866	0.020	0.023

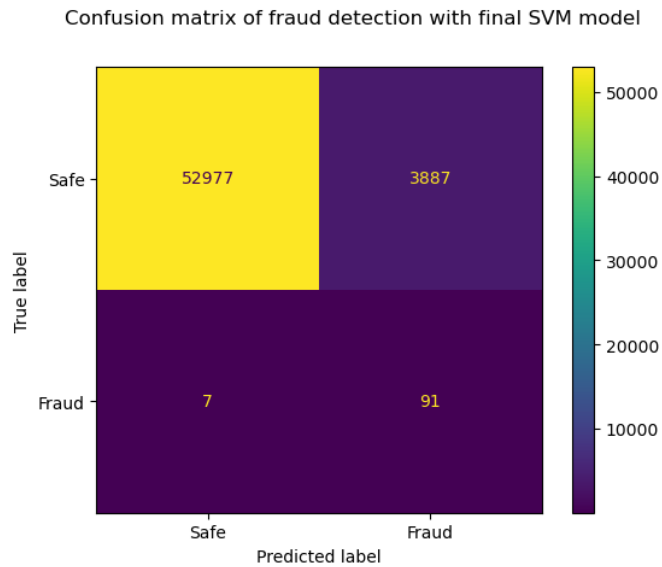
A-18: Tabes of comparison of training recall and validation recall across cross-validation folds.

	Training Recall	Validation Recall
Logistic Regression	0.928	0.904
SVM	0.987	0.929
Random Forest	0.999	0.866

A-19: Plots of expected costs vs. threshold.



A-20: Final confusion matrix.

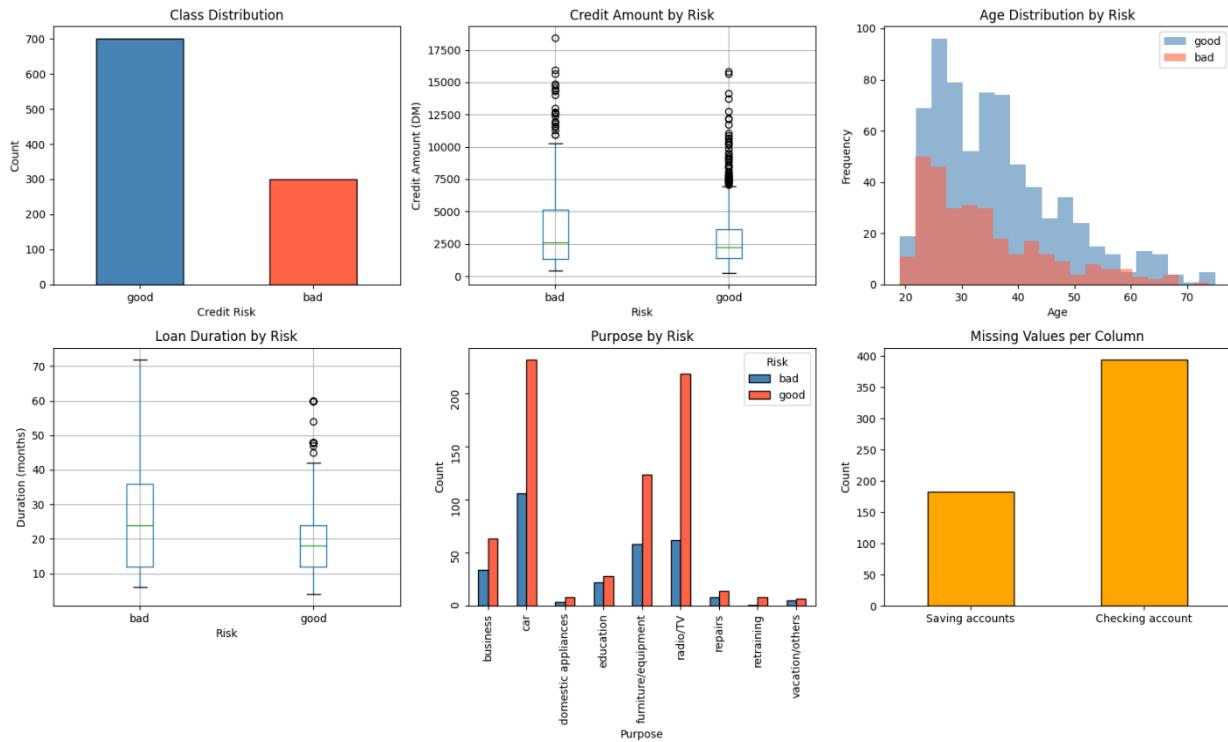


Appendix B

This section contains figures and code snippets that are related to credit card fraud detection tasks.

B-1: “EXPLORATORY DATA ANALYSIS (EDA)” for Credit dataset.

German Credit Dataset — EDA



B-2: Code for one hot encoding and train-test split for fraud dataset, and confirmation that class distributions are preserved for Credit dataset.

```
# — 4d. One-Hot Encoding —
# Converts text categories into binary columns.
# drop_first=True drops one dummy per feature to avoid
# multicollinearity (the dummy variable trap).
X_encoded = pd.get_dummies(X, columns=cat_cols, drop_first=True)

print(f"\nShape before encoding : {X.shape}")
print(f"Shape after encoding : {X_encoded.shape}")
print(f"\nEncoded columns:\n{X_encoded.columns.tolist()}")

# — 4e. Train-Test Split —
# stratify=y ensures both splits preserve the 70/30 class ratio

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\nTrain : {X_train.shape} → good: {(y_train==0).sum()} bad: {(y_train==1).sum()}")
print(f"Test : {X_test.shape} → good: {(y_test==0).sum()} bad: {(y_test==1).sum()}")

# — 4f. Feature Scaling —
# Tree models (RF, GB, XGB) don't need scaling but our
# Logistic Regression meta-learner does.
# IMPORTANT: fit only on train, transform both – never fit on test data.

scaler = StandardScaler()
X_train_scaled = pd.DataFrame(
    scaler.fit_transform(X_train), columns=X_train.columns
)
X_test_scaled = pd.DataFrame(
    scaler.transform(X_test), columns=X_test.columns
)
```

```

Shape before encoding : (1000, 9)
Shape after encoding  : (1000, 22)

Encoded columns:
['Age', 'Job', 'Credit amount', 'Duration', 'Sex_male', 'Housing_own', 'Housing_rent', 'Saving accounts_moderate', 'Saving accounts_none', 'Saving accounts_quite rich', 'Saving acco

Train : (800, 22)  →  good: 560  bad: 240
Test  : (200, 22)  →  good: 140  bad: 60

=== Scaling Done ===
Train mean (should be ~0): 0.0000
Train std  (should be ~1): 1.0006

```

B-3: Code for feature selection via Information Gain

```

def entropy(labels):
    n = len(labels)
    if n == 0:
        return 0.0
    counts = {}
    for l in labels:
        counts[l] = counts.get(l, 0) + 1
    return -sum((c / n) * np.log2(c / n) for c in counts.values() if c > 0)

def information_gain(X_col, y):
    """
    Computes IG between a continuous feature and binary target.
    We bin the feature into 10 equal-width bins to handle continuous values.
    """
    n = len(y)
    y = list(y)

    # Bin continuous feature into 10 equal-width bins
    col_min, col_max = min(X_col), max(X_col)
    n_bins = 10
    bin_width = (col_max - col_min) / n_bins if col_max != col_min else 1

    bins = []
    for v in X_col:
        b = int((v - col_min) / bin_width)
        b = min(b, n_bins - 1) # clamp last value into final bin
        bins.append(b)

    # H(Y) – entropy of target
    h_y = entropy(y)

    # H(Y|X) – weighted conditional entropy per bin
    h_y_given_x = 0.0
    for b in range(n_bins):

```

```

# H(Y) - entropy of target
h_y = entropy(y)

# H(Y|X) - weighted conditional entropy per bin
h_y_given_x = 0.0
for b in range(n_bins):
    subset_y = [y[i] for i in range(n) if bins[i] == b]
    if len(subset_y) == 0:
        continue
    h_y_given_x += (len(subset_y) / n) * entropy(subset_y)

return h_y - h_y_given_x

```

```

# --- Apply threshold of 0.01 ---
THRESHOLD = 0.01
top_features = ig_series[ig_series > THRESHOLD].index.tolist()

X_train_sel = X_train_scaled[top_features]
X_test_sel = X_test_scaled[top_features]

print("=== Selected Features ===")
for i, f in enumerate(top_features, 1):
    print(f" {i}. {f:40s} IG = {ig_series[f]:.4f}")

print(f"\nKept : {len(top_features)} features")
print(f"Dropped : {X_train_scaled.shape[1] - len(top_features)} features")
print(f"\nFinal train shape : {X_train_sel.shape}")
print(f"Final test shape : {X_test_sel.shape}")

```

```

9]
...
=== Selected Features ===
1. Checking account_none          IG = 0.0789
2. Duration                      IG = 0.0391
3. Credit amount                 IG = 0.0197
4. Saving accounts_none          IG = 0.0147
5. Housing_own                  IG = 0.0135
6. Age                          IG = 0.0132
7. Checking account_moderate     IG = 0.0104

```

```

Kept : 7 features
Dropped : 15 features

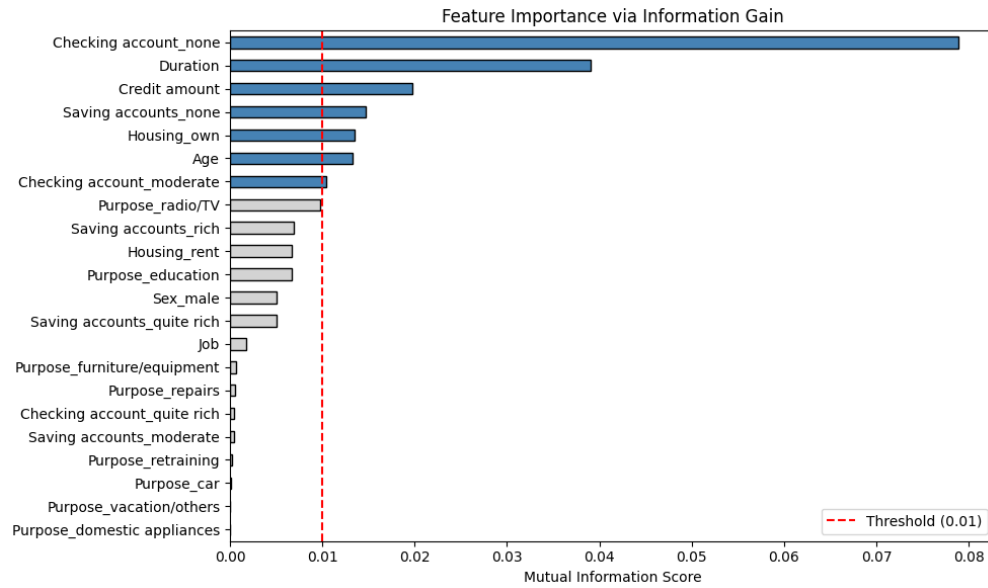
```

```

Final train shape : (800, 7)
Final test shape : (200, 7)

```

B-4: Plot of Feature Importance via Information Gain



B-5: Code for Logistic Regression Model

```
class LogisticRegression:
    def __init__(self, lr=0.01, n_iters=1000, random_state=42):
        self.lr = lr
        self.n_iters = n_iters
        self.random_state = random_state
        self.w = None
        self.b = None
        self.losses = []

    def _sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y, sample_weight=None):
        X = np.array(X)
        y = np.array(y)
        n, d = X.shape

        if sample_weight is None:
            sample_weight = np.ones(n)
            sample_weight = np.array(sample_weight, dtype=float)
            sample_weight = sample_weight / sample_weight.sum() * n

        self.w = np.zeros(d)
        self.b = 0.0

        for _ in range(self.n_iters):
            z = X @ self.w + self.b
            p = self._sigmoid(z)
            error = p - y

            dw = (1 / n) * (X.T @ (sample_weight * error))
            db = (1 / n) * (sample_weight * error).sum()

            self.w -= self.lr * dw
            self.b -= self.lr * db
```

```

        p_clip = np.clip(p, 1e-12, 1 - 1e-12)
        loss = -(1 / n) * (sample_weight * (
            y * np.log(p_clip) + (1 - y) * np.log(1 - p_clip)
        )).sum()
        self.losses.append(loss)

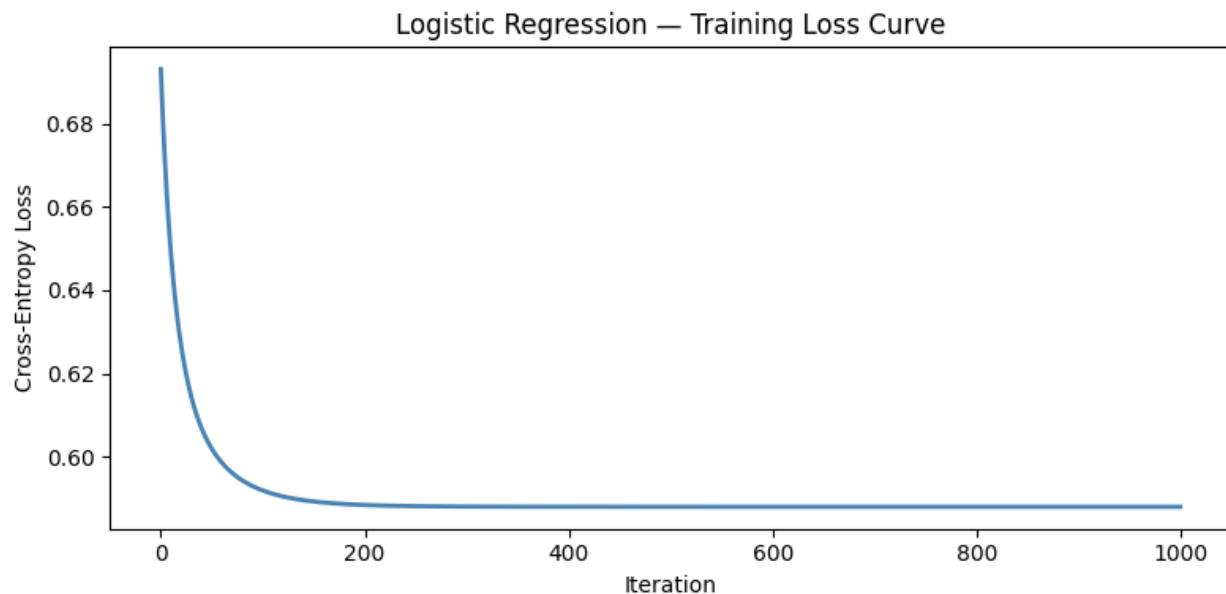
    return self

    def predict_proba(self, X):
        X = np.array(X)
        p = self._sigmoid(X @ self.w + self.b)
        return np.column_stack([1 - p, p])

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X)[:, 1] >= threshold).astype(int)

```

B-6: Plot of Logistic Regression - Training Loss Curve



B-7: Evaluation of Logistic Regression

```

--- Logistic Regression ---
Confusion Matrix → TP=41 TN=101 FP=39 FN=19
Accuracy : 0.7100
Precision : 0.5125
Recall : 0.6833
F1 Score : 0.5857
Cost : FN×5 + FP×1 = 19×5 + 39×1 = 134

```

B-8: Code for Decision Tree Model

```

# =====
# MODEL 2: DECISION TREE
# =====

class Node:
    """Represents a single node in the decision tree."""
    def __init__(self):
        self.feature = None # feature index used for split
        self.threshold = None # split threshold
        self.left = None # left child node
        self.right = None # right child node
        self.value = None # predicted class (leaf nodes only)
        self.proba = None # predicted probability (leaf nodes only)

class DecisionTree:
    def __init__(self, max_depth=10, min_samples_leaf=4,
                 criterion='gini', max_features=None, random_state=42):
        self.max_depth = max_depth
        self.min_samples_leaf = min_samples_leaf
        self.criterion = criterion # 'gini' or 'entropy'
        self.max_features = max_features # None = use all features
        self.random_state = random_state
        self.root = None
        self.rng = np.random.RandomState(random_state)

    def _impurity(self, y, sw):
        """Compute weighted Gini or Entropy for a node."""
        n = sw.sum()
        if n == 0:
            return 0.0
        p1 = (sw[y == 1]).sum() / n
        p0 = 1 - p1
        if self.criterion == 'gini':
            return 1 - p0**2 - p1**2
        else: # entropy
            eps = 1e-12
            p0 = max(p0, eps); p1 = max(p1, eps)
            return -(p0 * np.log2(p0) + p1 * np.log2(p1))

    def _best_split(self, X, y, sw):
        """Find the best feature and threshold to split on."""
        n, d = X.shape
        best_gain = -1
        best_feature = None
        best_threshold = None

        parent_impurity = self._impurity(y, sw)

        # Feature subsampling (used by Random Forest)
        if self.max_features is None:
            features = list(range(d))
        else:
            k = max(1, int(self.max_features * d))
            features = list(self.rng.choice(d, k, replace=False))

        for f in features:
            col = X[:, f]
            thresholds = np.unique(col)

            for t in thresholds:
                left_mask = col <= t
                right_mask = ~left_mask

```

```

        if left_mask.sum() < self.min_samples_leaf:
            continue
        if right_mask.sum() < self.min_samples_leaf:
            continue

        sw_l = sw[left_mask]; sw_r = sw[right_mask]
        y_l = y[left_mask]; y_r = y[right_mask]

        n_l = sw_l.sum(); n_r = sw_r.sum()
        n_t = sw.sum()

        gain = parent_impurity \
            - (n_l / n_t) * self._impurity(y_l, sw_l) \
            - (n_r / n_t) * self._impurity(y_r, sw_r)

        if gain > best_gain:
            best_gain = gain
            best_feature = f
            best_threshold = t

    return best_feature, best_threshold

def _build(self, X, y, sw, depth):
    """Recursively build the tree."""
    node = Node()

    # Weighted majority class for leaf prediction
    w1 = sw[y == 1].sum()
    w0 = sw[y == 0].sum()
    node.value = 1 if w1 >= w0 else 0
    node.proba = w1 / (w0 + w1) if (w0 + w1) > 0 else 0

    # Stop conditions
    if depth >= self.max_depth:
        return node
    if len(y) < 2 * self.min_samples_leaf:
        return node
    if len(np.unique(y)) == 1:
        return node

    feat, thresh = self._best_split(X, y, sw)

    if feat is None:
        return node

    left_mask = X[:, feat] <= thresh
    right_mask = ~left_mask

    node.feature = feat
    node.threshold = thresh
    node.left = self._build(X[left_mask], y[left_mask],
                            sw[left_mask], depth + 1)
    node.right = self._build(X[right_mask], y[right_mask],
                             sw[right_mask], depth + 1)
    return node

```

```

def fit(self, X, y, sample_weight=None):
    X = np.array(X)
    y = np.array(y)
    n = len(y)
    if sample_weight is None:
        sample_weight = np.ones(n)
    sw = np.array(sample_weight, dtype=float)
    self.root = self._build(X, y, sw, depth=0)
    return self

def _predict_row(self, row, node):
    """Traverse tree for a single sample."""
    if node.feature is None:
        return node.value, node.proba
    if row[node.feature] <= node.threshold:
        return self._predict_row(row, node.left)
    else:
        return self._predict_row(row, node.right)

def predict(self, X):
    X = np.array(X)
    return np.array([self._predict_row(row, self.root)[0] for row in X])

def predict_proba(self, X):
    X = np.array(X)
    probas = np.array([self._predict_row(row, self.root)[1] for row in X])
    return np.column_stack([1 - probas, probas])

```

B-9: Evaluation of Decision Tree Model

```

--- Decision Tree ---
Confusion Matrix → TP=39 TN=88 FP=52 FN=21
Accuracy : 0.6350
Precision : 0.4286
Recall : 0.6500
F1 Score : 0.5166
Cost : FN×5 + FP×1 = 21×5 + 52×1 = 157

```

B-10: Code for Random Forest Model

```

class RandomForest:
    def __init__(self, n_estimators=100, max_depth=10,
                  min_samples_leaf=4, criterion='gini',
                  max_features='sqrt', random_state=42):
        self.n_estimators = n_estimators
        self.max_depth = max_depth
        self.min_samples_leaf = min_samples_leaf
        self.criterion = criterion
        self.max_features = max_features # 'sqrt' or float
        self.random_state = random_state
        self.trees = []
        self.rng = np.random.RandomState(random_state)

```

```

def _bootstrap(self, X, y, sw):
    """Sample n rows with replacement (bootstrap)."""
    n = len(y)
    idx = self.rng.choice(n, n, replace=True)
    return X[idx], y[idx], sw[idx]

def _get_max_features(self, d):
    if self.max_features == 'sqrt':
        return max(1, int(np.sqrt(d))) / d
    elif isinstance(self.max_features, float):
        return self.max_features
    return 1.0 # use all features

def fit(self, X, y, sample_weight=None):
    X = np.array(X)
    y = np.array(y)
    n = len(y)
    if sample_weight is None:
        sample_weight = np.ones(n)
    sw = np.array(sample_weight, dtype=float)

    d = X.shape[1]
    max_feats = self._get_max_features(d)
    self.trees = []

    for i in range(self.n_estimators):
        tree = DecisionTree(
            max_depth = self.max_depth,
            min_samples_leaf = self.min_samples_leaf,
            criterion = self.criterion,
            max_features = max_feats,
            random_state = self.random_state + i
        )
        X_b, y_b, sw_b = self._bootstrap(X, y, sw)
        tree.fit(X_b, y_b, sample_weight=sw_b)
        self.trees.append(tree)

        if (i + 1) % 25 == 0:
            print(f" {i + 1}/{self.n_estimators} trees trained")

    return self

def predict_proba(self, X):
    X = np.array(X)
    all_probas = np.array([t.predict_proba(X[:, 1]) for t in self.trees])
    avg_proba = all_probas.mean(axis=0)
    return np.column_stack([1 - avg_proba, avg_proba])

def predict(self, X, threshold=0.5):
    return (self.predict_proba(X)[:, 1] >= threshold).astype(int)

```

B-11: Evaluation of Random Forest Model

```

--- Random Forest ---
Confusion Matrix → TP=36 TN=104 FP=36 FN=24
Accuracy : 0.7000
Precision : 0.5000
Recall : 0.6000
F1 Score : 0.5455
Cost : FN×5 + FP×1 = 24×5 + 36×1 = 156

```

B-12: Code for Gradient Boosting

```
class GradientBoosting:
    def __init__(self, n_estimators=100, learning_rate=0.05,
                 max_depth=4, min_samples_leaf=4,
                 subsample=0.8, random_state=42):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.max_depth = max_depth
        self.min_samples_leaf = min_samples_leaf
        self.subsample = subsample
        self.random_state = random_state
        self.trees = []
        self.F0 = None
        self.rng = np.random.RandomState(random_state)

    def _sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def _log_odds(self, y):
        """Initial prediction: log(p / (1-p)) where p = mean(y)."""
        p = np.mean(y)
        p = np.clip(p, 1e-12, 1 - 1e-12)
        return np.log(p / (1 - p))

    def fit(self, X, y, sample_weight=None):
        X = np.array(X)
        y = np.array(y, dtype=float)
        n = len(y)

        if sample_weight is None:
            sample_weight = np.ones(n)
        sw = np.array(sample_weight, dtype=float)

        self.F0 = self._log_odds(y)
        F = np.full(n, self.F0)

        self.trees = []
```

```
        for m in range(self.n_estimators):
            p = self._sigmoid(F)
            r = y - p
            n_sub = max(1, int(self.subsample * n))
            sub_idx = self.rng.choice(n, n_sub, replace=False)
            X_sub = X[sub_idx]
            r_sub = r[sub_idx]
            sw_sub = sw[sub_idx]

            tree = DecisionTree(
                max_depth = self.max_depth,
                min_samples_leaf = self.min_samples_leaf,
                criterion = 'gini',
                random_state = self.random_state + m
            )
            r_binary = (r_sub > 0).astype(int)
            tree.fit(X_sub, r_binary, sample_weight=sw_sub)

            self._set_leaf_values(tree.root, X_sub, r_sub, sw_sub)
```

```

        F += self.learning_rate * self._predict_raw(tree.root, X)
        self.trees.append(tree)

        if (m + 1) % 25 == 0:
            loss = -np.mean(y * np.log(np.clip(self._sigmoid(F), 1e-12, 1))
                            + (1-y) * np.log(np.clip(1 - self._sigmoid(F), 1e-12, 1)))
            print(f" [{m+1}/{self.n_estimators}] loss = {loss:.4f}")

    return self

def _set_leaf_values(self, node, X, r, sw):
    """Override leaf node values with weighted mean of residuals."""
    if node.feature is None:
        w_sum = sw.sum()
        node.proba = (sw * r).sum() / w_sum if w_sum > 0 else 0.0
        return
    mask_l = X[:, node.feature] <= node.threshold
    mask_r = ~mask_l
    if mask_l.sum() > 0:
        self._set_leaf_values(node.left, X[mask_l], r[mask_l], sw[mask_l])
    if mask_r.sum() > 0:
        self._set_leaf_values(node.right, X[mask_r], r[mask_r], sw[mask_r])

def _predict_raw_row(self, row, node):
    if node.feature is None:
        return node.proba
    if row[node.feature] <= node.threshold:
        return self._predict_raw_row(row, node.left)
    else:
        return self._predict_raw_row(row, node.right)

def _predict_raw(self, root, X):
    return np.array([self._predict_raw_row(row, root) for row in X])

def predict_proba(self, X):
    X = np.array(X)
    F = np.full(len(X), self.F0)
    for tree in self.trees:
        F += self.learning_rate * self._predict_raw(tree.root, X)
    p = self._sigmoid(F)
    return np.column_stack([1 - p, p])

def predict(self, X, threshold=0.5):
    return (self.predict_proba(X)[:, 1] >= threshold).astype(int)

```

B-13: Evaluation of Gradient Boosting

```

--- Gradient Boosting ---
Confusion Matrix → TP=35 TN=108 FP=32 FN=25
Accuracy : 0.7150
Precision : 0.5224
Recall : 0.5833
F1 Score : 0.5512
Cost : FN×5 + FP×1 = 25×5 + 32×1 = 157

```

B-14: Code for XGBoost Model


```

class XGBNode:
    """Node for XGBoost tree - stores optimal leaf weights."""
    def __init__(self):
        self.feature = None
        self.threshold = None
        self.left = None
        self.right = None
        self.weight = 0.0 # optimal leaf weight w*

class XGBoost:
    def __init__(self, n_estimators=100, learning_rate=0.05,
                 max_depth=4, min_samples_leaf=4,
                 subsample=0.8, colsample=0.8,
                 reg_lambda=1.0, reg_gamma=0.0,
                 scale_pos_weight=1.0, random_state=42):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.max_depth = max_depth
        self.min_samples_leaf = min_samples_leaf
        self.subsample = subsample
        self.colsample = colsample
        self.reg_lambda = reg_lambda
        self.reg_gamma = reg_gamma
        self.scale_pos_weight = scale_pos_weight
        self.random_state = random_state
        self.trees = []
        self.F0 = None
        self.rng = np.random.RandomState(random_state)

    def _sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def _gradients(self, y, F):
        p = self._sigmoid(F)
        g = p - y
        h = p * (1 - p)
        scale = np.where(y == 1, self.scale_pos_weight, 1.0)
        return g * scale, h * scale

    def _leaf_weight(self, G, H):
        return -G / (H + self.reg_lambda)

    def _split_gain(self, G_L, H_L, G_R, H_R):
        lam = self.reg_lambda
        return 0.5 * (
            G_L**2 / (H_L + lam) +
            G_R**2 / (H_R + lam) -
            (G_L + G_R)**2 / (H_L + H_R + lam)
        ) - self.reg_gamma

```

```

def _split_gain(self, G_L, H_L, G_R, H_R):
    lam = self.reg_lambda
    return 0.5 * (
        G_L**2 / (H_L + lam) +
        G_R**2 / (H_R + lam) -
        (G_L + G_R)**2 / (H_L + H_R + lam)
    ) - self.reg_gamma

def _best_split(self, X, g, h, features):
    best_gain = 0.0
    best_feat = None
    best_thresh = None

    for f in features:
        col = X[:, f]
        thresholds = np.unique(col)

        for t in thresholds:
            left = col <= t
            right = ~left

            if left.sum() < self.min_samples_leaf:
                continue
            if right.sum() < self.min_samples_leaf:
                continue

            G_L, H_L = g[left].sum(), h[left].sum()
            G_R, H_R = g[right].sum(), h[right].sum()
            gain = self._split_gain(G_L, H_L, G_R, H_R)

            if gain > best_gain:
                best_gain = gain
                best_feat = f
                best_thresh = t

    return best_feat, best_thresh

def _build(self, X, g, h, depth, features):
    node = XGBNode()
    G = g.sum(); H = h.sum()
    node.weight = self._leaf_weight(G, H)

    if depth >= self.max_depth:
        return node
    if len(g) < 2 * self.min_samples_leaf:
        return node

    feat, thresh = self._best_split(X, g, h, features)
    if feat is None:
        return node

    left_mask = X[:, feat] <= thresh
    right_mask = ~left_mask

    node.feature = feat
    node.threshold = thresh
    node.left = self._build(X[left_mask], g[left_mask],
                            h[left_mask], depth + 1, features)
    node.right = self._build(X[right_mask], g[right_mask],
                             h[right_mask], depth + 1, features)
    return node

```

```

def fit(self, X, y, sample_weight=None):
    X = np.array(X)
    y = np.array(y, dtype=float)
    n = len(y)
    d = X.shape[1]

    p0 = np.clip(np.mean(y), 1e-12, 1 - 1e-12)
    self.F0 = np.log(p0 / (1 - p0))
    F = np.full(n, self.F0)
    self.trees = []

    for m in range(self.n_estimators):
        g, h = self._gradients(y, F)

        n_sub = max(1, int(self.subsample * n))
        row_idx = self.rng.choice(n, n_sub, replace=False)

        n_col = max(1, int(self.colsample * d))
        col_idx = list(self.rng.choice(d, n_col, replace=False))

        X_sub = X[row_idx][:, col_idx]
        g_sub = g[row_idx]
        h_sub = h[row_idx]

        root = self._build(X_sub, g_sub, h_sub, depth=0,
                           features=list(range(len(col_idx))))
        root_mapped = self._remap_features(root, col_idx)

        F += self.learning_rate * self._predict_raw(X, root_mapped)
        self.trees.append(root_mapped)

        if (m + 1) % 25 == 0:
            p_c = np.clip(self._sigmoid(F), 1e-12, 1 - 1e-12)
            loss = -np.mean(y * np.log(p_c) + (1 - y) * np.log(1 - p_c))
            print(f" [{m+1}/{self.n_estimators}] loss = {loss:.4f}")

    return self

def predict_proba(self, X):
    X = np.array(X)
    F = np.full(len(X), self.F0)
    for root in self.trees:
        F += self.learning_rate * self._predict_raw(X, root)
    p = self._sigmoid(F)
    return np.column_stack([1 - p, p])

def predict(self, X, threshold=0.5):
    return (self.predict_proba(X)[:, 1] >= threshold).astype(int)

```

B-15: Evaluation of XGBoost Model

```

--- XGBoost ---
Confusion Matrix → TP=39 TN=100 FP=40 FN=21
Accuracy : 0.6950
Precision : 0.4937
Recall : 0.6500
F1 Score : 0.5612
Cost : FN×5 + FP×1 = 21×5 + 40×1 = 145

```

B-16: Code for Stacked Classifier Model

```

kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

base_models = [('rf', rf), ('gb', gb), ('xgb', xgb)]

# — Layer 1: Generate out-of-fold predictions —————
train_meta = np.zeros((X_train_sel.shape[0], len(base_models)))

for i, (name, model) in enumerate(base_models):
    oof_preds = np.zeros(X_train_sel.shape[0])

    for fold, (train_idx, val_idx) in enumerate(kf.split(X_train_sel, y_train)):
        X_fold_train = X_train_sel.iloc[train_idx]
        y_fold_train = y_train.iloc[train_idx]
        X_fold_val = X_train_sel.iloc[val_idx]
        sw_fold = sample_weights[train_idx]

        model.fit(X_fold_train, y_fold_train, sample_weight=sw_fold)
        oof_preds[val_idx] = model.predict_proba(X_fold_val)[:, 1]

    train_meta[:, i] = oof_preds
    print(f"✓ {name} out-of-fold predictions done")

print(f"\nMeta-feature matrix shape: {train_meta.shape}") # expect (800, 3)

# — Re-train base models on full training set —————
for name, model in base_models:
    model.fit(X_train_sel, y_train, sample_weight=sample_weights)
print("\n✓ Base models retrained on full training data")

# — Layer 2: Train meta-learner —————
meta_learner = LogisticRegression(lr=0.1, n_iters=1000, random_state=42)
meta_learner.fit(train_meta, y_train, sample_weight=sample_weights)
print("✓ Meta-learner (Logistic Regression) trained")

print("\n=== Meta-learner Weights ===")
for (name, _), coef in zip(base_models, meta_learner.w):
    print(f" {name:5s} : {coef:.4f}")

# — Generate test meta-features —————
test_meta = np.column_stack([
    model.predict_proba(X_test_sel)[:, 1]
    for _, model in base_models
])

# — Final predictions from meta-learner —————
y_pred_stack = meta_learner.predict(test_meta)
y_proba_stack = meta_learner.predict_proba(test_meta)[:, 1]

```

B-17: Evaluation of Stacked Ensemble

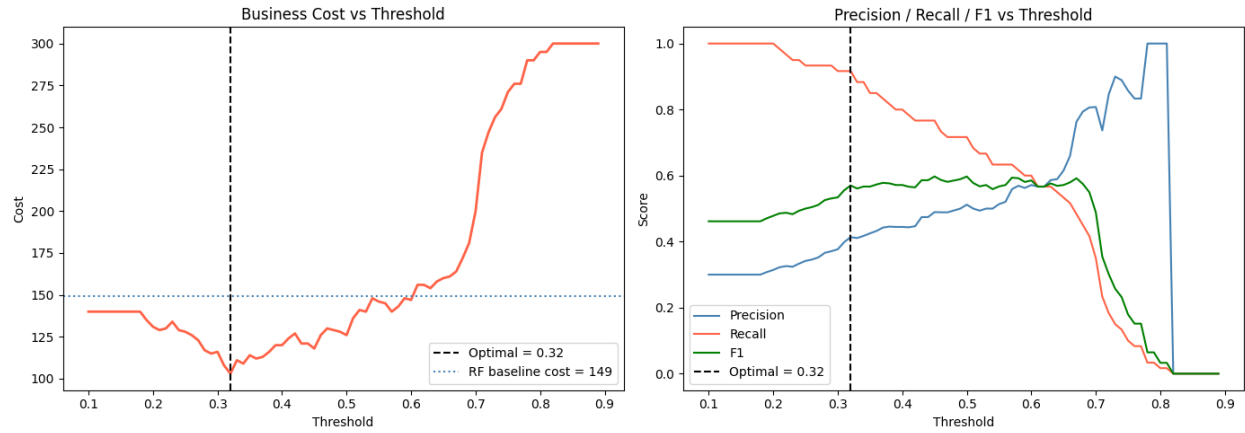
```

=== Meta-learner Weights ===
rf      : 1.3263
gb      : 0.8789
xgb     : 1.6799

--- Stacked Ensemble ---
Confusion Matrix → TP=43 TN=99 FP=41 FN=17
Accuracy   : 0.7100
Precision  : 0.5119
Recall     : 0.7167
F1 Score   : 0.5972
Cost       : FN×5 + FP×1 = 17×5 + 41×1 = 126

```

B-18: Plot of Decision Threshold Tuning



B-19: AUC + Type I & Type II Error Analysis

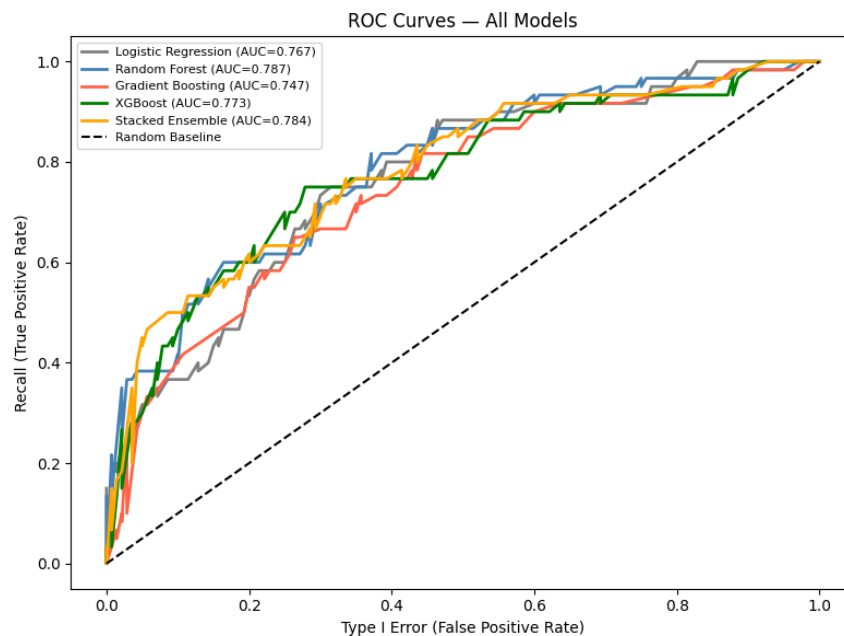
```

=== AUC Scores ===
Logistic Regression      : AUC = 0.7672
Random Forest           : AUC = 0.7868
Gradient Boosting        : AUC = 0.7470
XGBoost                 : AUC = 0.7728
Stacked Ensemble         : AUC = 0.7840

=== Type I & Type II Error Analysis ===
Model                    Type I (FPR)  Type II (FNR)  AUC
-----
Logistic Regression      0.2786       0.3167        0.7672
Random Forest            0.2786       0.3667        0.7868
Gradient Boosting        0.2286       0.4167        0.7470
XGBoost                  0.2714       0.3000        0.7728
Stacked Ensemble         0.5571       0.0833        0.7840

```

B-20: ROC Curve for all the Models



B-21: 5-Fold Cross-Validation Results for all Model

```
--- Logistic Regression (threshold=0.5) ---  
accuracy   : 0.6838 ± 0.0406  
precision  : 0.4802 ± 0.0487  
recall     : 0.6792 ± 0.0867  
f1         : 0.5623 ± 0.0617  
cost       : 112.2000 ± 22.6429
```

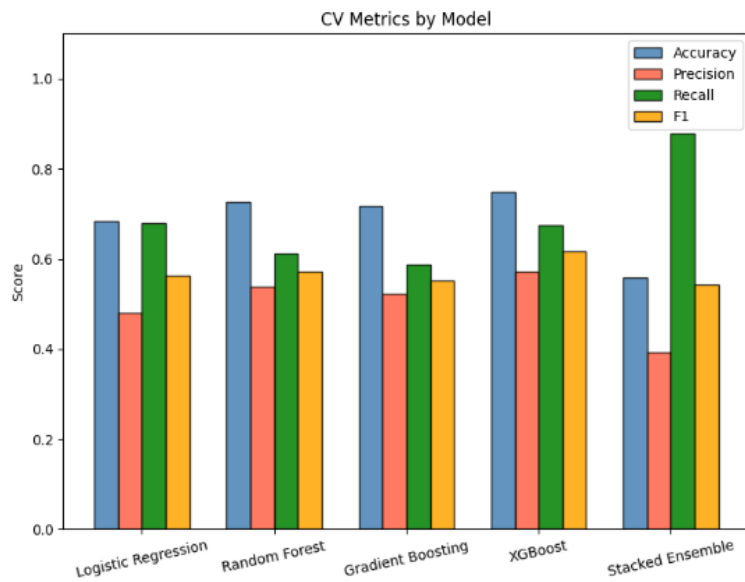
```
--- Random Forest (threshold=0.5) ---  
accuracy   : 0.7263 ± 0.0338  
precision  : 0.5389 ± 0.0514  
recall     : 0.6125 ± 0.0635  
f1         : 0.5729 ± 0.0535  
cost       : 118.2000 ± 17.0059
```

```
--- Gradient Boosting (threshold=0.5) ---  
accuracy   : 0.7175 ± 0.0307  
precision  : 0.5223 ± 0.0456  
recall     : 0.5875 ± 0.1182  
f1         : 0.5511 ± 0.0780  
cost       : 124.4000 ± 27.2727
```

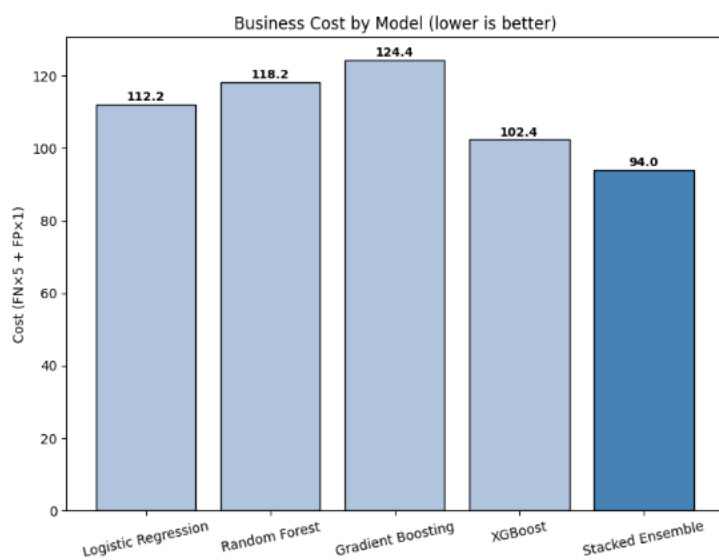
```
--- XGBoost (threshold=0.5) ---  
accuracy   : 0.7500 ± 0.0426  
precision  : 0.5711 ± 0.0628  
recall     : 0.6750 ± 0.0841  
f1         : 0.6178 ± 0.0672  
cost       : 102.4000 ± 22.0749
```

```
--- Stacked Ensemble (threshold=0.32) ---  
accuracy   : 0.5575 ± 0.0298  
precision  : 0.3932 ± 0.0257  
recall     : 0.8792 ± 0.0851  
f1         : 0.5431 ± 0.0398  
cost       : 94.0000 ± 20.2361
```

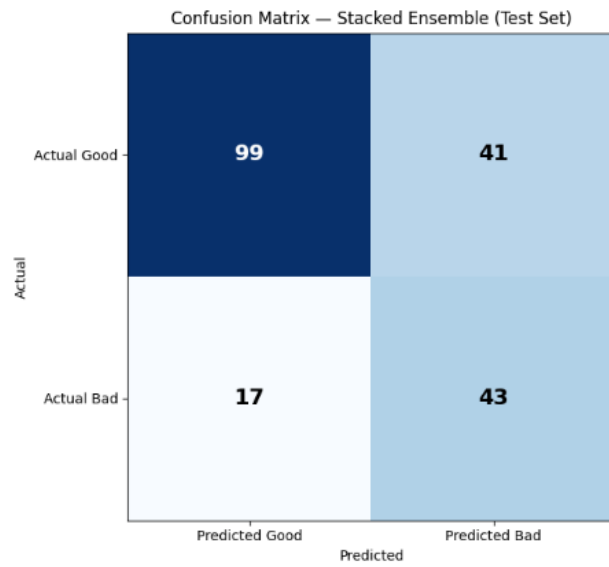
B-22: CV Metrics by Model



B-23: Business Cost by Model



B-24: Confusion Matrix for Stacked Ensemble



B-25: Recall vs Business cost comparison for all models

